

창의 공학 설계 - 스팸 필터
최종 발표

컴퓨터 공학과 20230546 서보경, 20230794 이상민, 20230490 박채원

목차

1. 프로젝트 목적 및 목표
2. 관련 기술 현황
3. 개발 동기
4. 개발 내용 – AI
5. 개발 내용 – ios
6. 개발 내용 – Android

프로젝트 목적

iMessage
5월 20일 (화) 오후 2:14

퇴근 후 지친 하루에 달콤한 휴식이 필요
하신가요?
가까운 곳에서 찾고 싶으신가요?
섹시하고 매력적인 여성과 특별한
시간을 보내고 싶다면
원하시는 스타일의 사진을 보내드릴테니
제 라인추가해 주세요 :auua8778

iMessage
5월 8일 (목) 오후 1:07

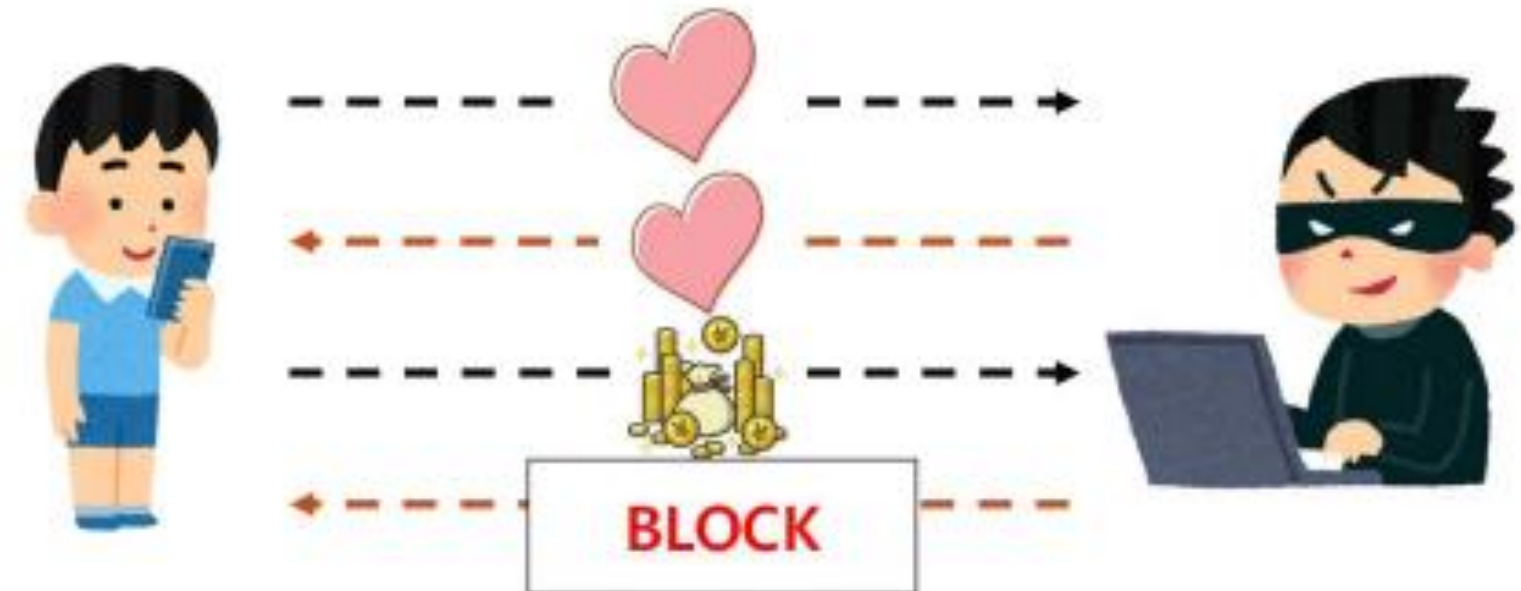
22살 대학생이에요. 한국에 유학온
지 얼마 안됐는데 친구들이랑 같이
알바로 용돈 벌고 있거든요. 근처에 있고
바로 출장 가능해요. 음주 폭력 절대 안
됩니다. 저희 사진 보고 싶으시면 개인톡
보내주시고요. 토이 가져오셔서
같이 놀아도 돼요. 상황 봐서 하룻밤
보내도 된답니다.추가해 주세요내 LINE:
bck869

프로젝트 목적

스팸 필터링 앱이 없다면?



스팸 필터링 앱이 없다면?



문제 인식

프로젝트 목표

기능 1 - 로컬 스팸 차단 (on)



기능 2 - 고급 스팸 차단 (on)



해결 방안 마련

관련 기술 및 현황

< 검색



스마트 스팸 필터: 똑똑한
AI가 잡아주는 스팸 차...

AI 연구자가 만든 똑똑한 AI 스팸/...



207개의 평가

4.6

★★★★★

연령

17+

세

카테고리



유틸리티

개발:



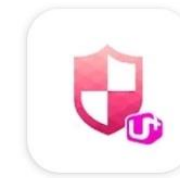
Junbun

새로운 소식 >

버전 1.7.2

1주 전

- Upgraded Ondevice model (3x data volume)



U+스팸차단

LG유플러스(LG Uplus Corporation)



3.5★

리뷰 1만개 ①

500만회 이상

다운로드

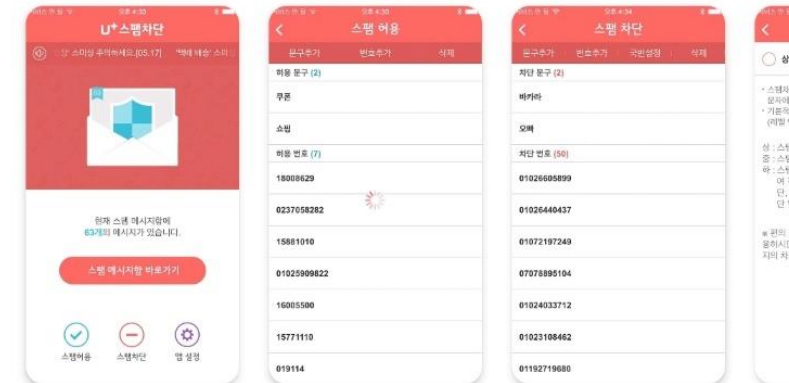
3

3세 이상 ①

설치



휴대전화에 설치, 더 많은 기기에 설치 가능



앱 정보

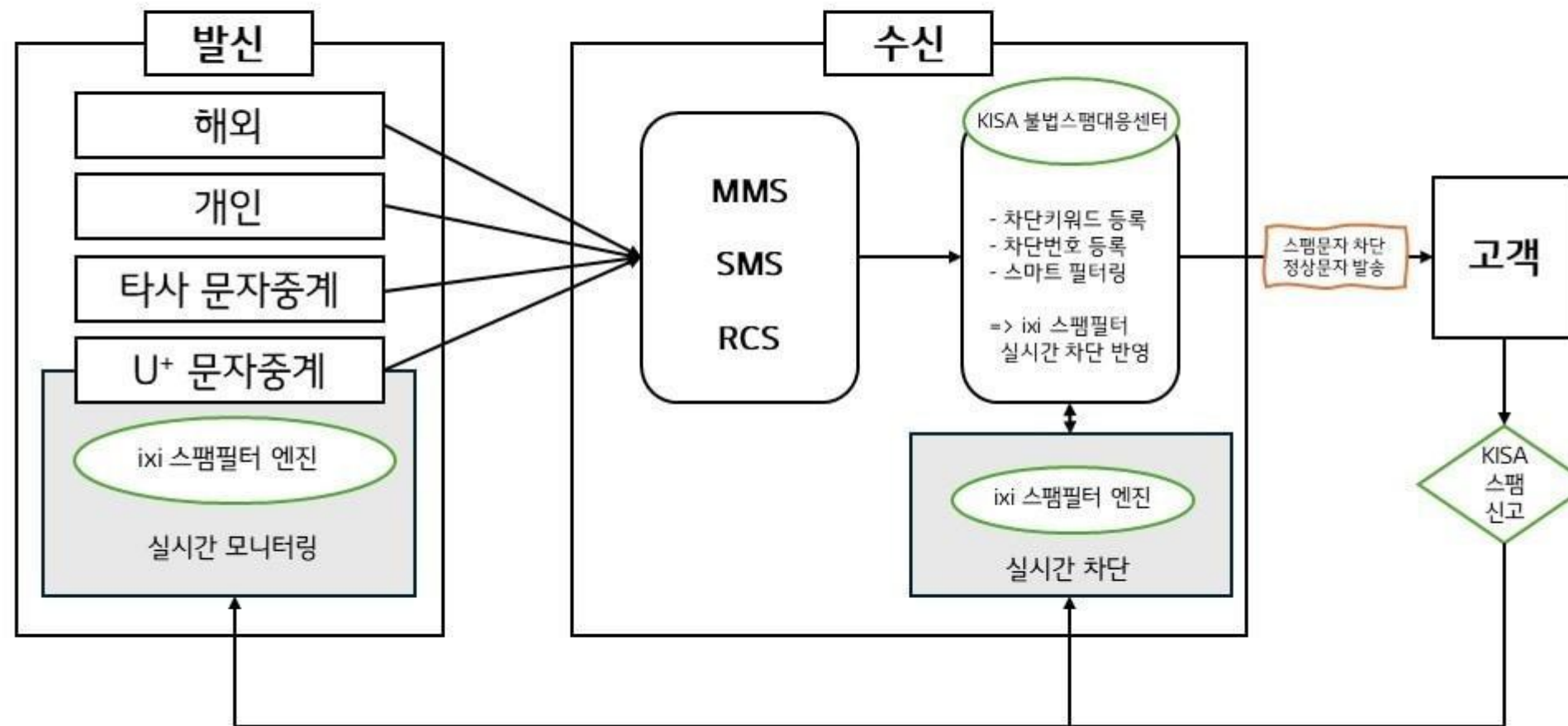


스팸차단APP은 고객에게 수신되는 각종 스팸을 막아주는, 유플러스가 제공하는 무료 스팸 차단 어플입니다.

상용화된 스팸 필터

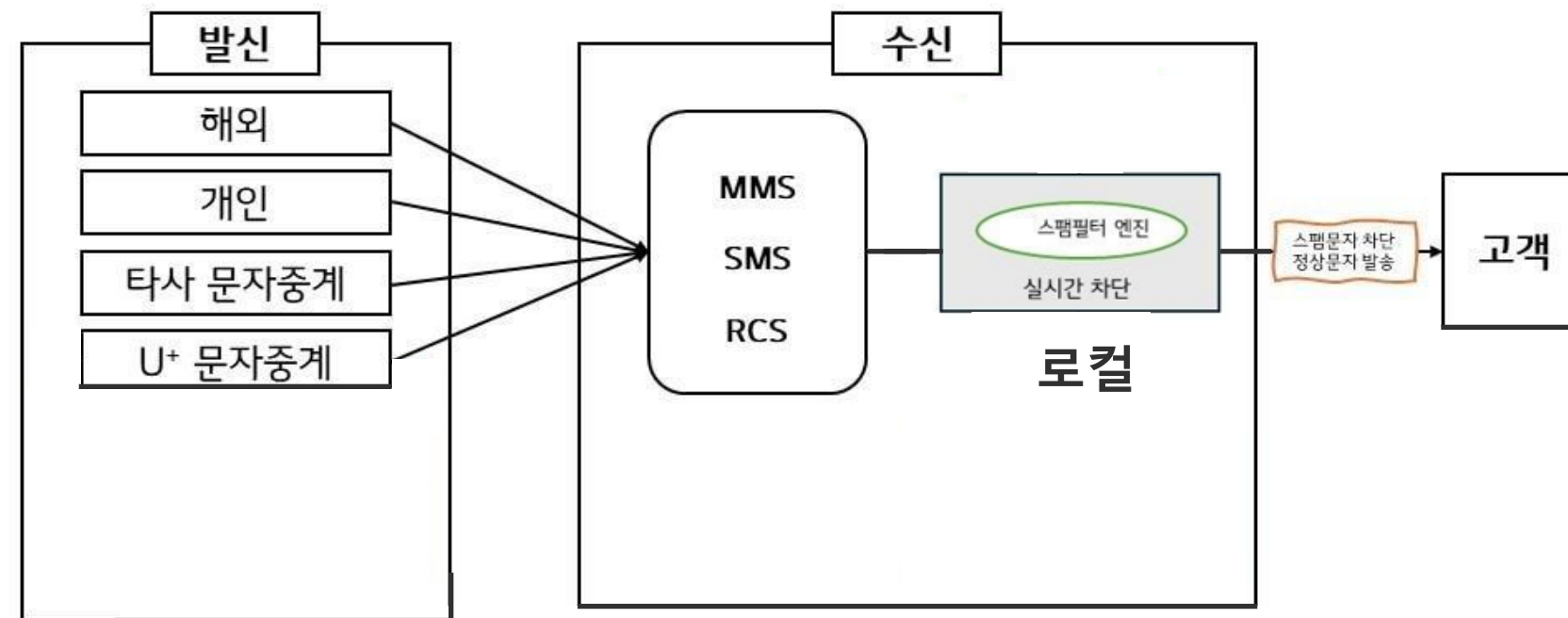
관련 기술 및 현황

ixi 스팸필터

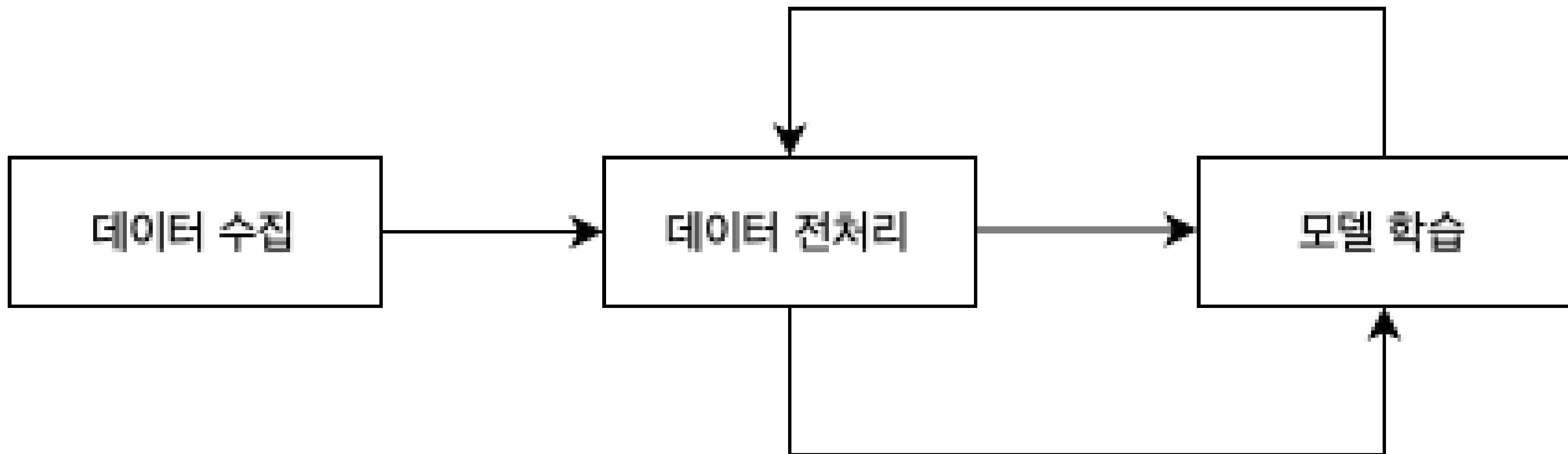


일반적인 스팸 필터 구조

삼 초 고 려 스팸필터



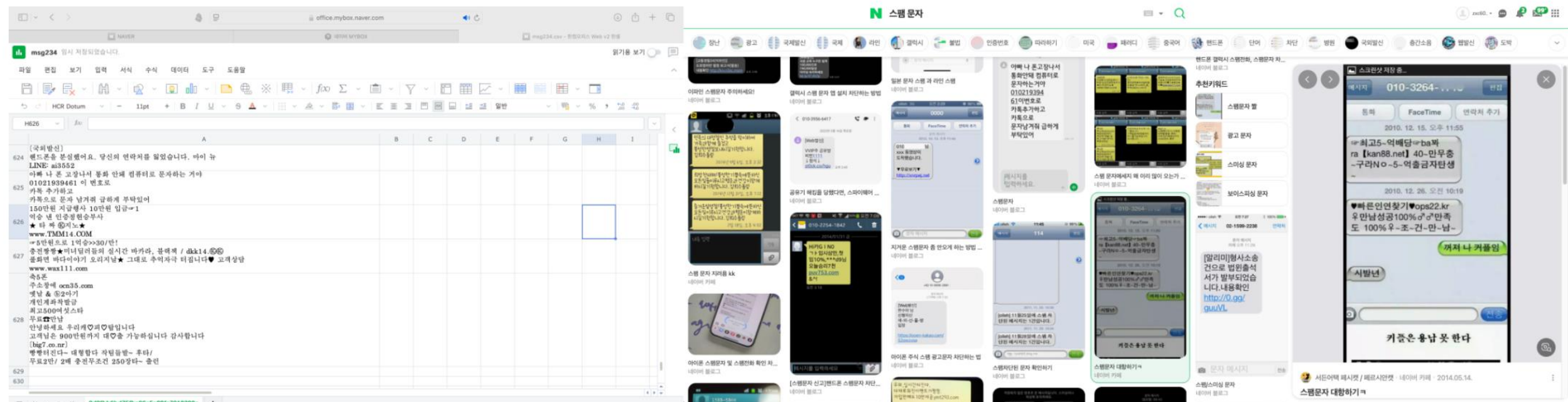
삼초고려식 스팸 필터 구조

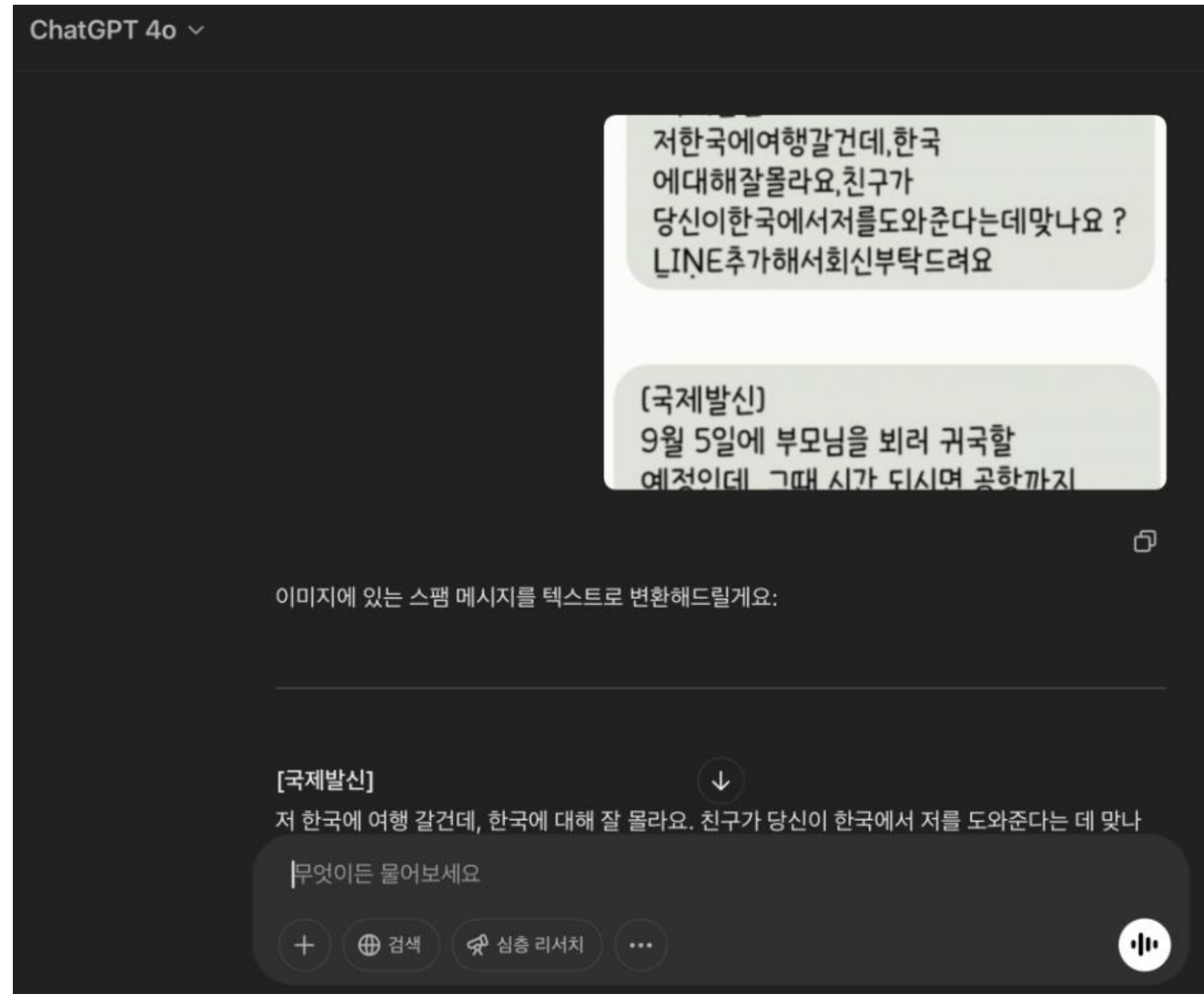


Backup Viewer				
Easy Check	/HomeDomain/Library/SMS			
App List (186)	  View  Export  Refresh			
Common Directory	File Name	Modified Date	Type	Size
App Document	 Drafts		Folder	--
Camera Roll	 sms.db	2025-03-26 20:13...	db File	14.21 MB
Contacts				
Call History				
Messages				
Notes				
Voice Memos				
Safari Bookmarks				
Desktop				
Settings				

스팸 데이터 수집

3/26 수





기 존 t r a i n s e t	
Spam	540
Ham	360

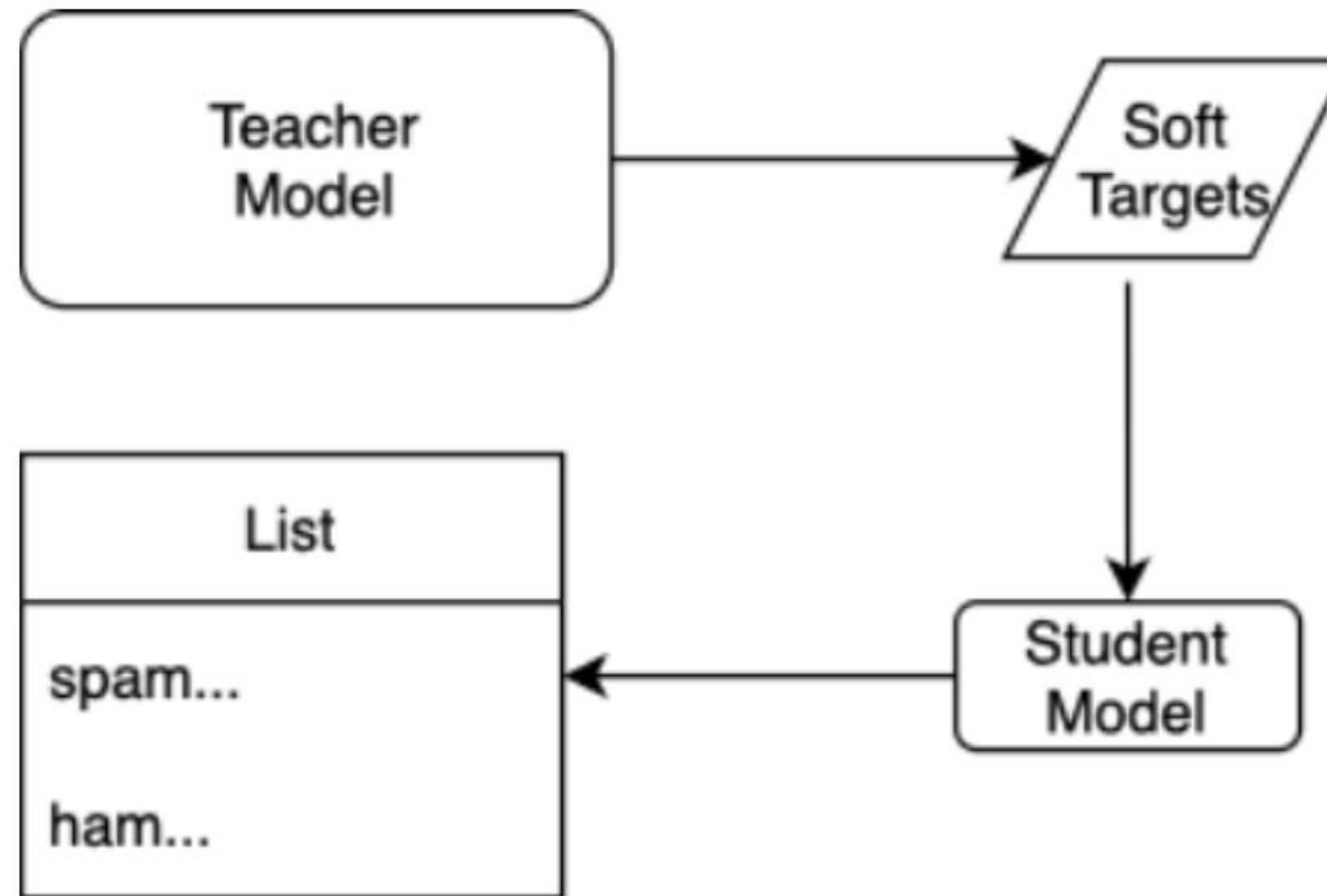
새 로 운 t r a i n s e t	
Spam	650
Ham	4532

AI 데이터 전처리

데이터 전처리를 위한 아이디어

- 라벨을 0과 1로 표현하면 ai가 잘 학습하지 못함
- > 스팸일 확률, 리스크 5단계, 스팸 여부 + 스팸인 이유 등등으로 라벨링

그 외 불용어 제거, 정규화 (w_n , " ", 숫자->NUM...)



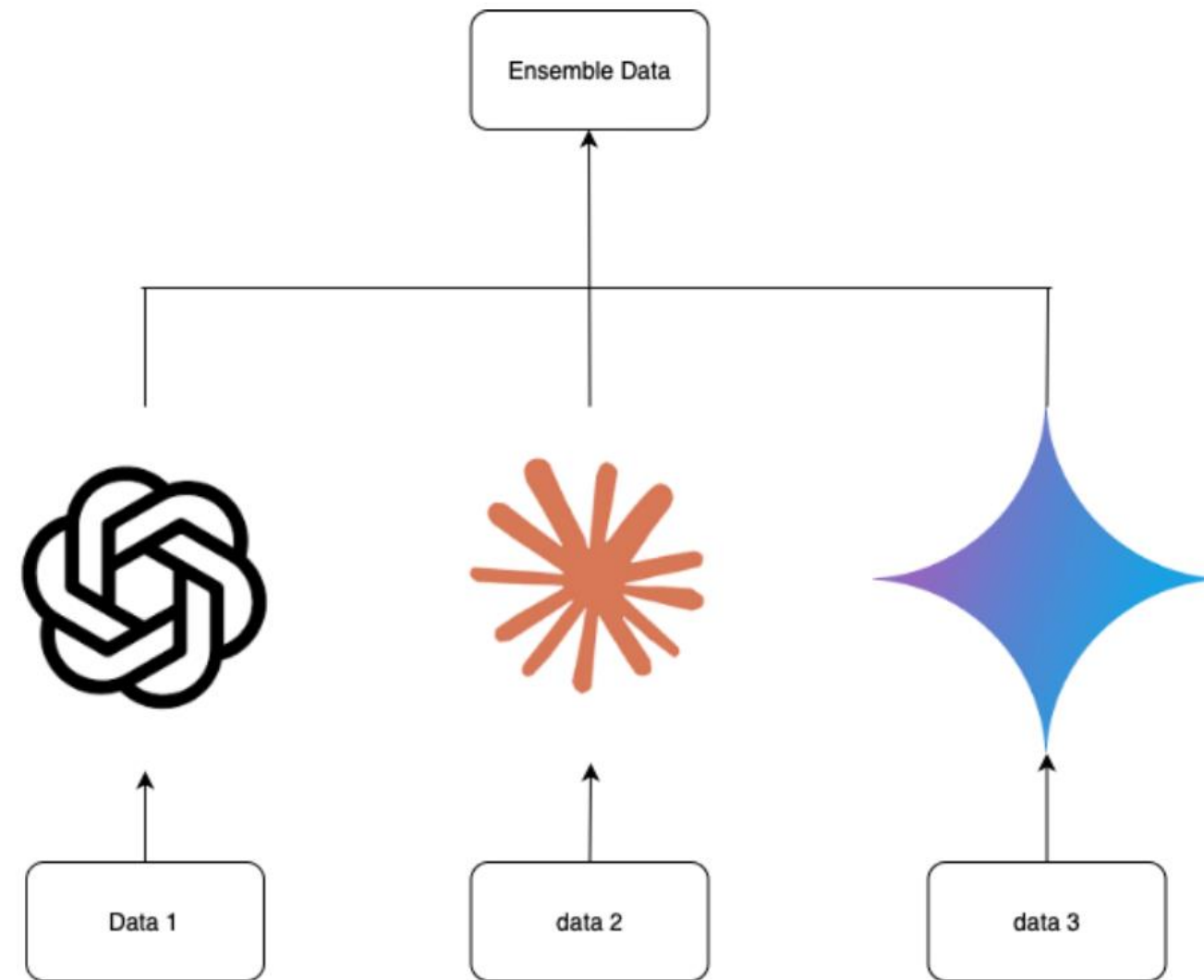
teacher 모델을 이용한 라벨링

```
"id": 1,  
"text": "퇴근 후에 좀 쉬고 싶으신가요?  
가까이에 있는      찾고 싶으신가요?  
섹시하고 아름다운 여성과 즐거운 데이트를 하고 싶으신가요?  
고르실 사진 보내드릴테니 제 라인 추가해 주세요 : ap6638",  
"source": "4o"  
}
```

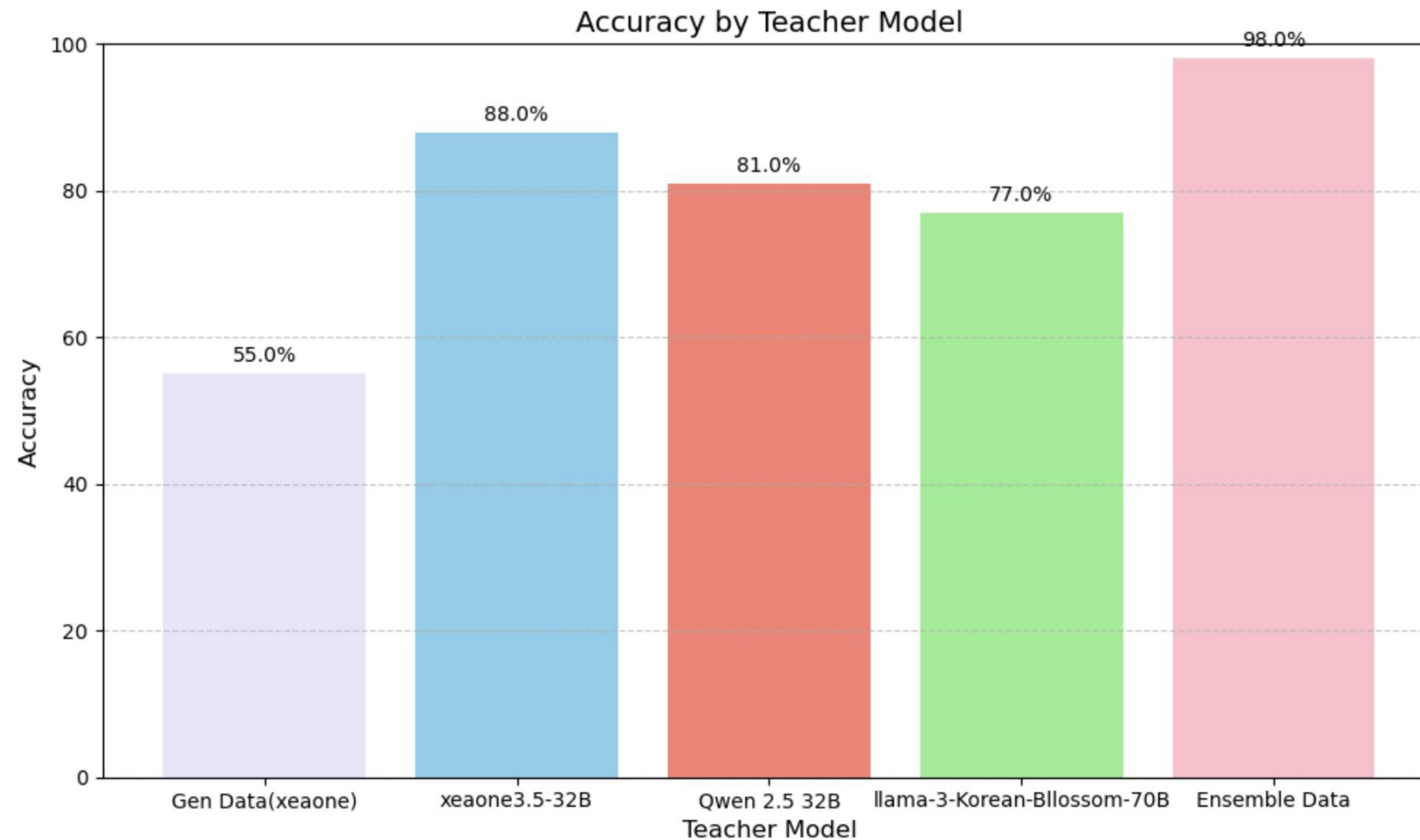
json 복사 편집

```
{  
  "id": 1,  
  "label": "Highly Suspicious"  
}
```

있는건 그냥 쓰자...



세상에서 가장 강한 teacher



각 teacher 별 성능 / EXAONE 3.5

데이터 전처리

{ 각종 정규화를 위한 전처리 : 스팸일 확률 }

-> {문자 : 스팸일 확률} 쌍으로 데이터 포맷을 결정



EXAONE 3.5

LG AI Research's Large scale models:
expert-level performance for
professional and enterprise tasks

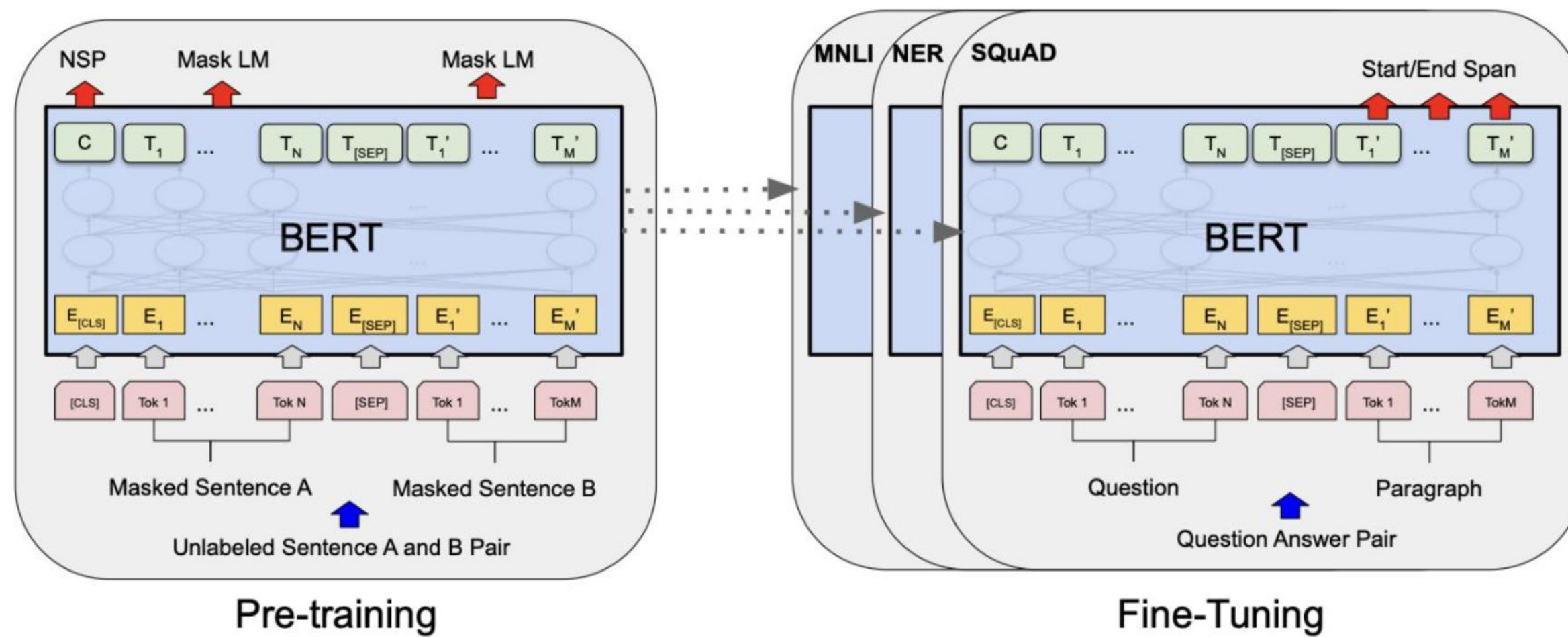
조건에 맞는 llm

데이터 전처리

LLM (GPT)	0.83	0.77	0.80	0.76	0.88
Random Forest	0.80	0.70	0.75	0.70	
Gradient Boosting	0.78	0.68	0.73	0.68	

굳이 llm을 써야하는 이유?

이전 발표 중에서....



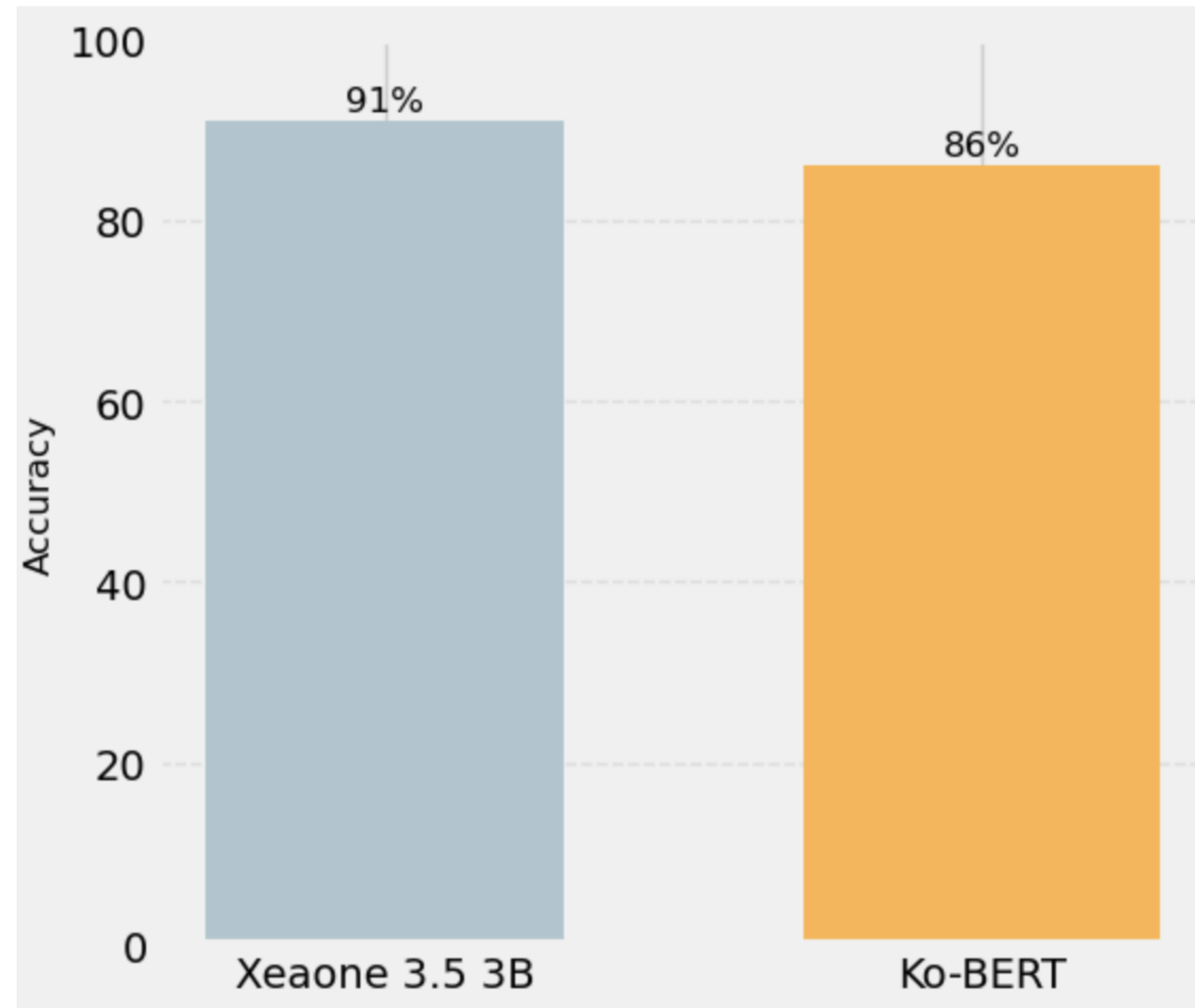
XEAONE -> BERT

XEAONE -> BERT

약 5%정도의 정확도 차이

10배 이상 가벼워진 모델

-> 타협 가능한 수준이라고 판단



Model card	Files and versions	Community	Settings
main	kosp	Go to file	1 contributor
History: 6 commits			
sangmin12312343 Fix vocab size mismatch d1941c8 VERIFIED			
.gitattributes	Safe	1.52 kB	initial commit
config.json		667 Bytes	Fix vocab size mismatch
model.safetensors		436 MB LFS	Fix vocab size mismatch
optimizer.pt	pickle	738 MB LFS	Upload KoBERT spam filter
rng_state.pth	pickle	14.2 kB LFS	Upload KoBERT spam filter
scheduler.pt	Safe pickle	1.06 kB LFS	Upload KoBERT spam filter
special_tokens_map.json	Safe	695 Bytes	Fix vocab size mismatch
tokenizer.json		721 kB	Upload tokenizer.json with huggingface_hub
tokenizer_config.json	Safe	1.27 kB	Upload tokenizer_config.json with huggingface_...
trainer_state.json		1.35 kB	Upload KoBERT spam filter
training_args.bin	pickle	5.24 kB LFS	Upload KoBERT spam filter

변환 후 배포

Framework

Core ML

Integrate machine learning models into your app.

iOS 11.0+ | iPadOS 11.0+ | Mac Catalyst 13.0+ | macOS 10.13+ | tvOS 11.0+ | visionOS 1.0+ | watchOS 4.0+

Overview

Use [Core ML](#) to integrate machine learning models into your app. [Core ML](#) provides a unified representation for all models. Your app uses [Core ML](#) APIs and user data to make predictions, and to train or fine-tune models, all on a person's device.



변환 후 배포

온디바이스 스팸 필터링 iOS 앱 구현

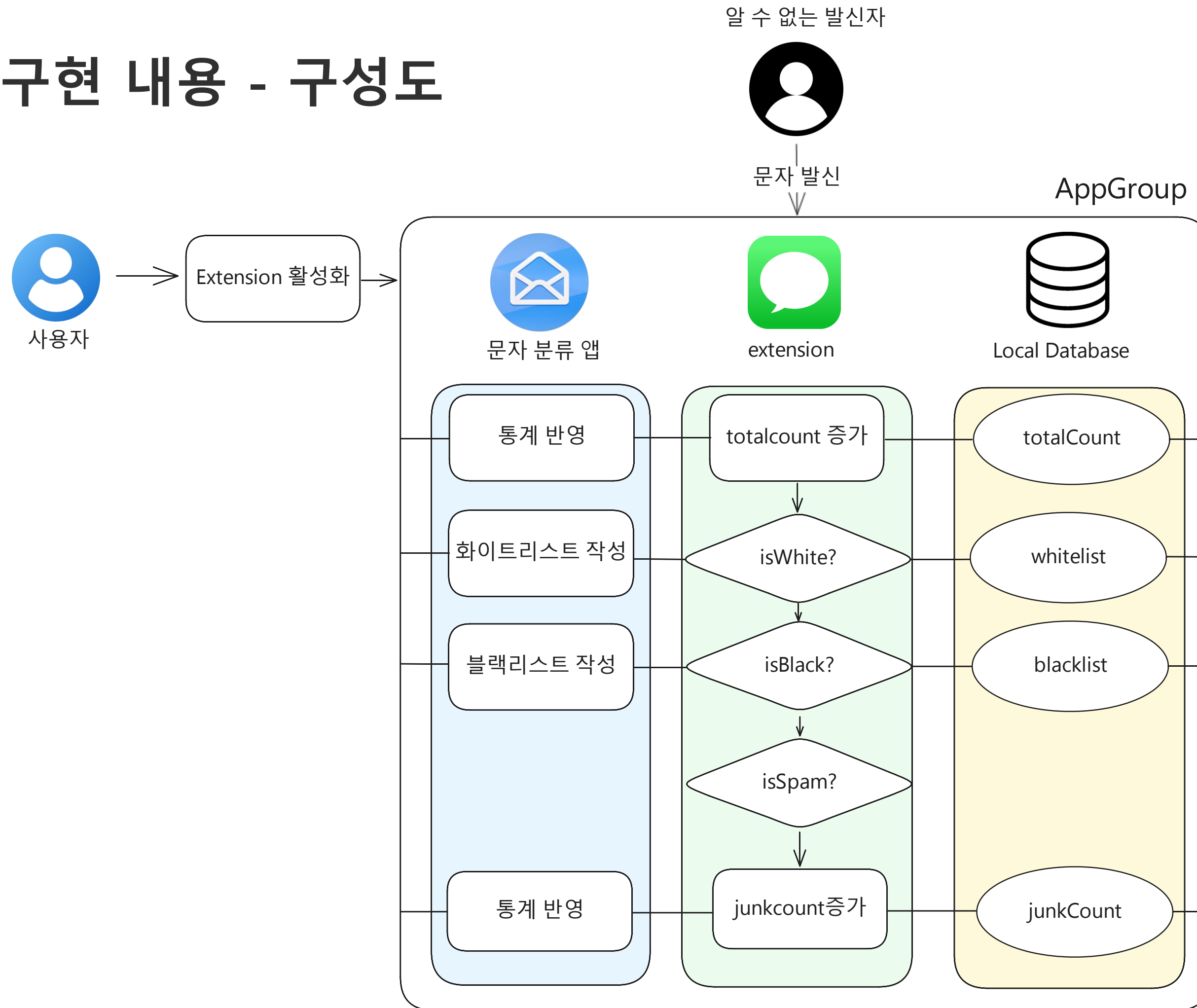
iOS 목표

1. Ui 구현
2. 블랙리스트, 화이트리스트 구현
3. Extension을 통해 문자 수신 시 실시간 필터링 적용
4. Ui와 extension 연결
 1. 블랙/화이트리스트 연결
 2. 통계 연결
5. Ai 동작을 위한 Tokenizer 작성
6. Ai과 Extension 연동
7. Ai 성능 개선

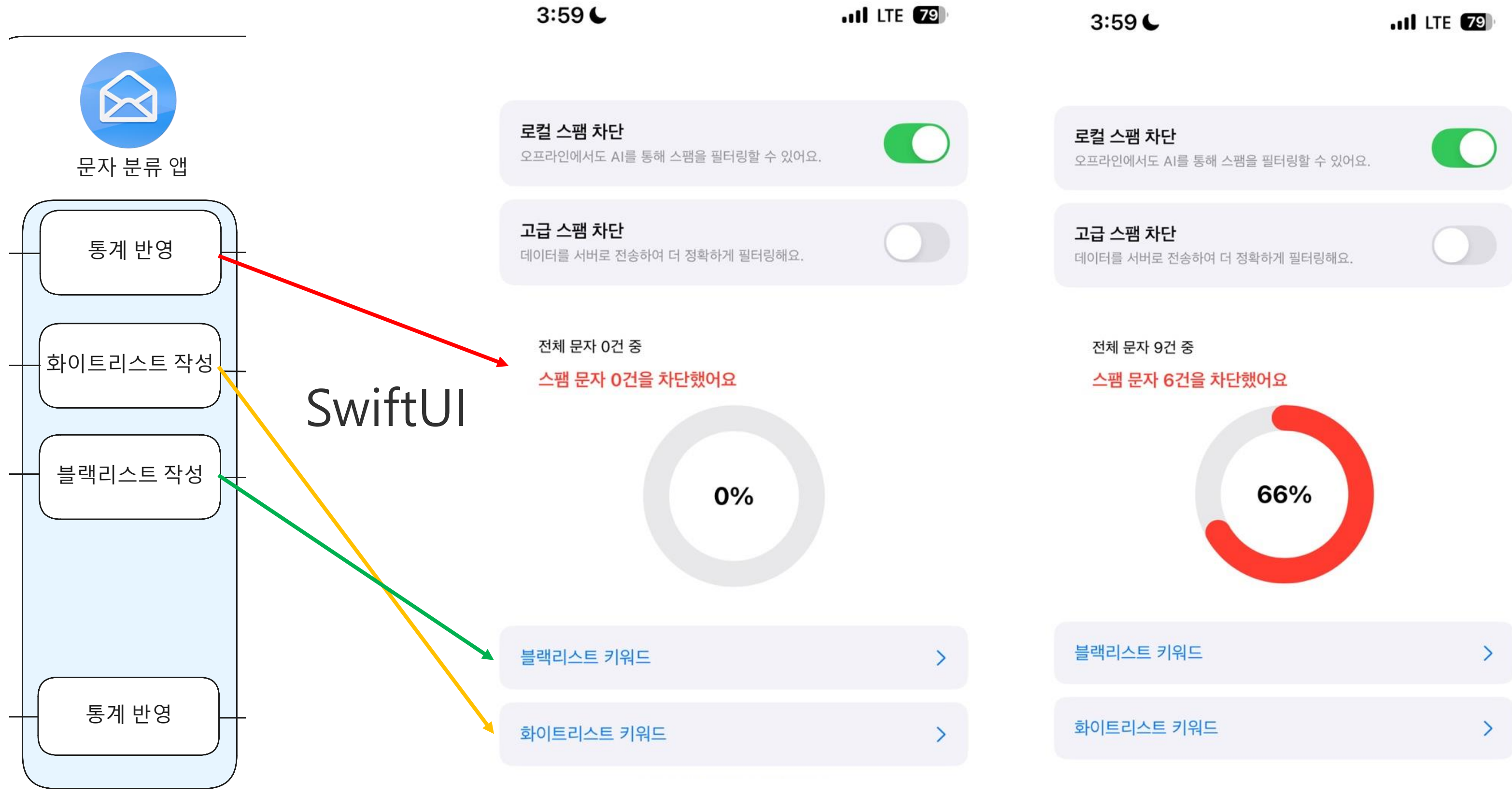
iOS 목표

1. Ui 구현 
2. 블랙리스트, 화이트리스트 구현 
3. Extension을 통해 문자 수신 시 실시간 필터링 적용 
4. Ui와 extension 연결
 1. 블랙/화이트리스트 연결 
 2. 통계 연결 
5. Ai 동작을 위한 Tokenizer 작성 
6. Ai과 Extension 연동 
7. Ai 성능 개선 

iOS 주요 구현 내용 - 구성도



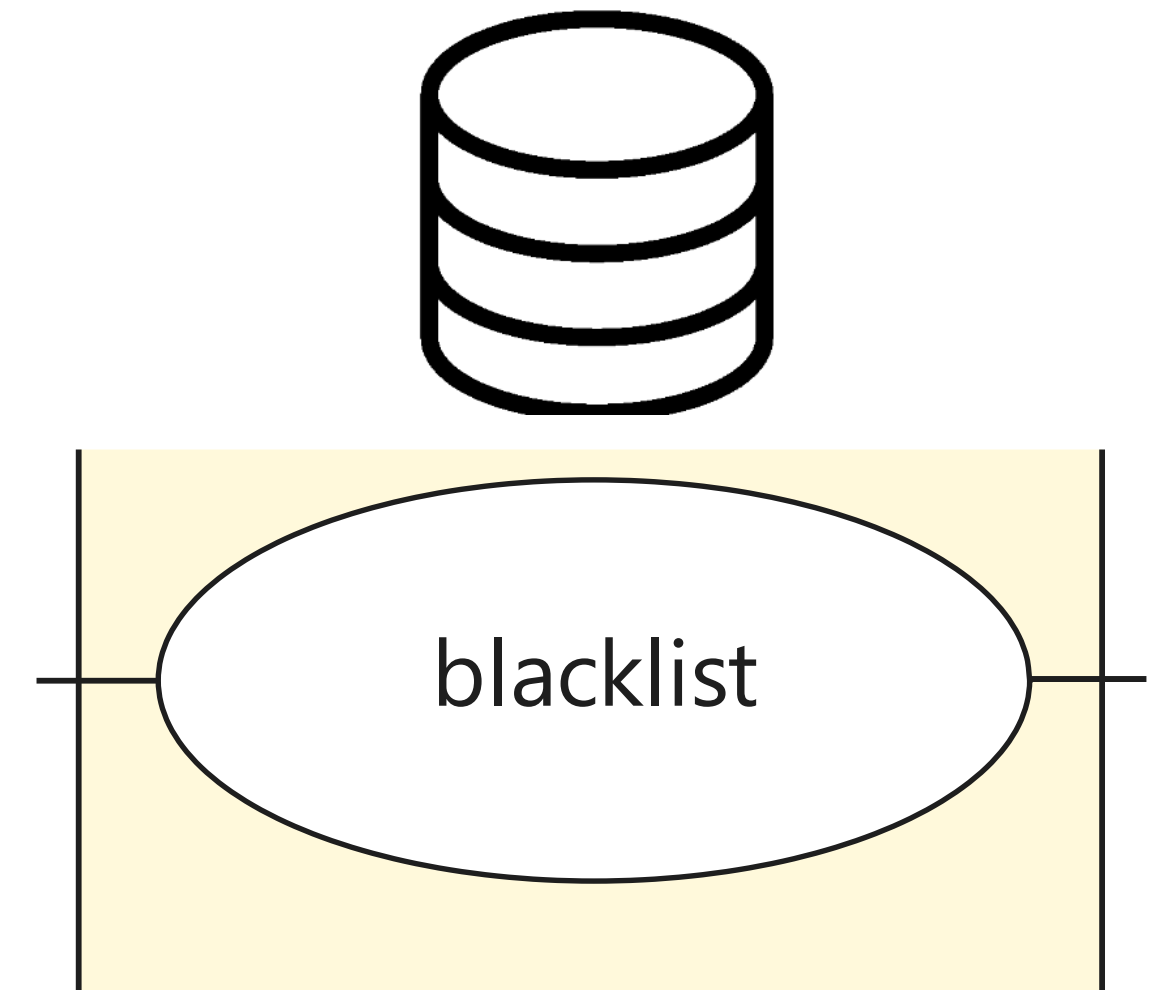
iOS 주요 구현 내용 - 기술요소



iOS 주요 구현 내용 - 기술요소



로컬에 저장
(UserDefaults)



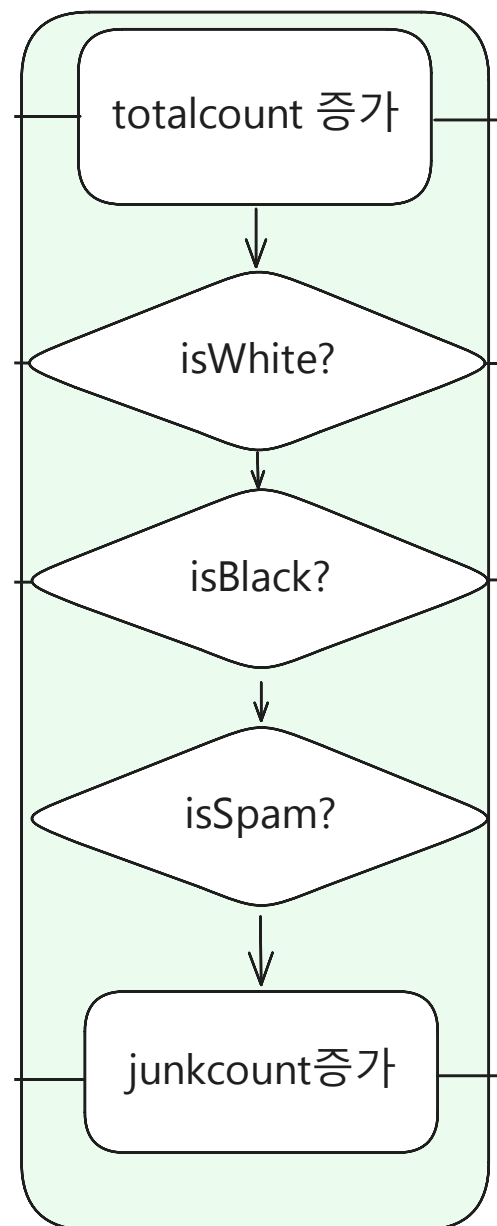
```
func saveKeywords() {  
    let defaults = UserDefaults(suiteName: "group.com.messagefilterapp.shared")  
    defaults?.set(keywords, forKey: "blacklist")  
}  
  
func loadKeywords() {  
    let defaults = UserDefaults(suiteName: "group.com.messagefilterapp.shared")  
    keywords = defaults?.stringArray(forKey: "blacklist") ?? []  
}
```

왼쪽으로 드래그 시
삭제

iOS 주요 구현 내용 - 기술요소



extension



Message
Filter Extension

ILMessageFilterExtension 사용

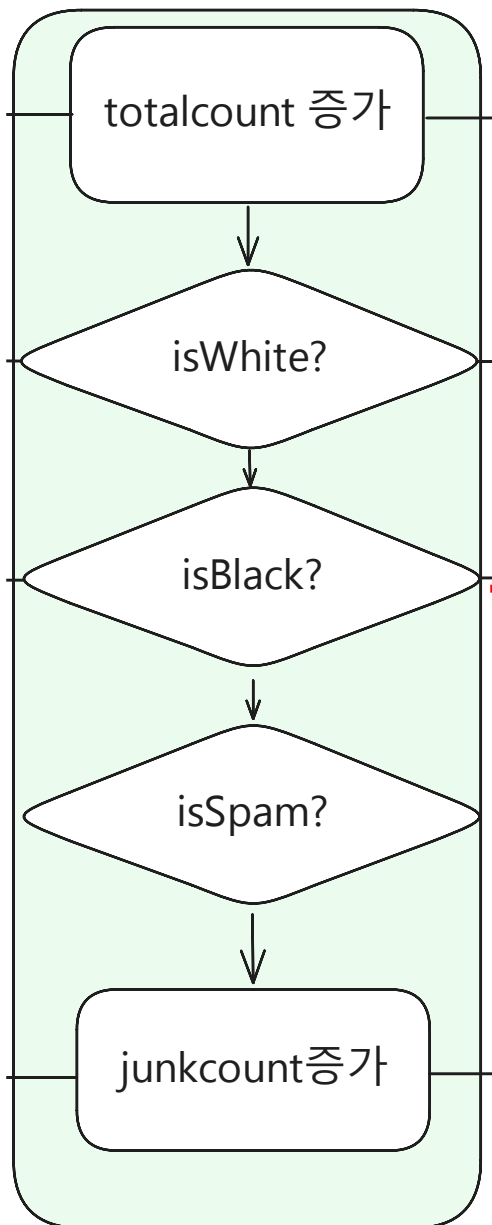
```
func handle(_ queryRequest: ILMessageFilterQueryRequest,  
            context: ILMessageFilterExtensionContext,  
            completion: @escaping (ILMessageFilterQueryResponse) -> Void)
```

```
private func offlineAction(  
    for queryRequest: ILMessageFilterQueryRequest  
) -> (ILMessageFilterAction, ILMessageFilterSubAction)
```

iOS 주요 구현 내용 - 기술요소



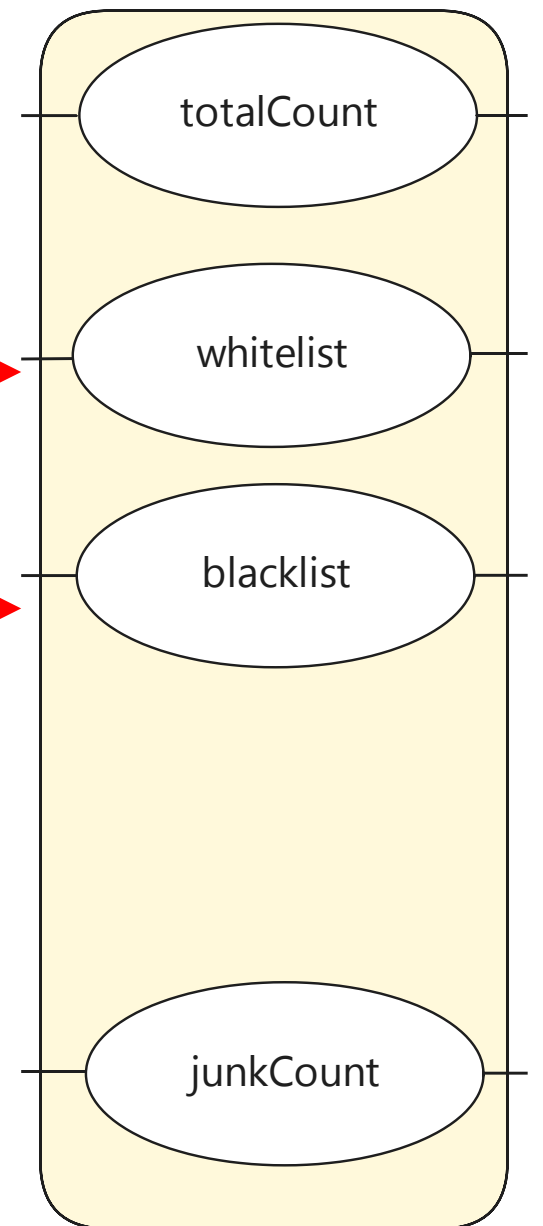
extension



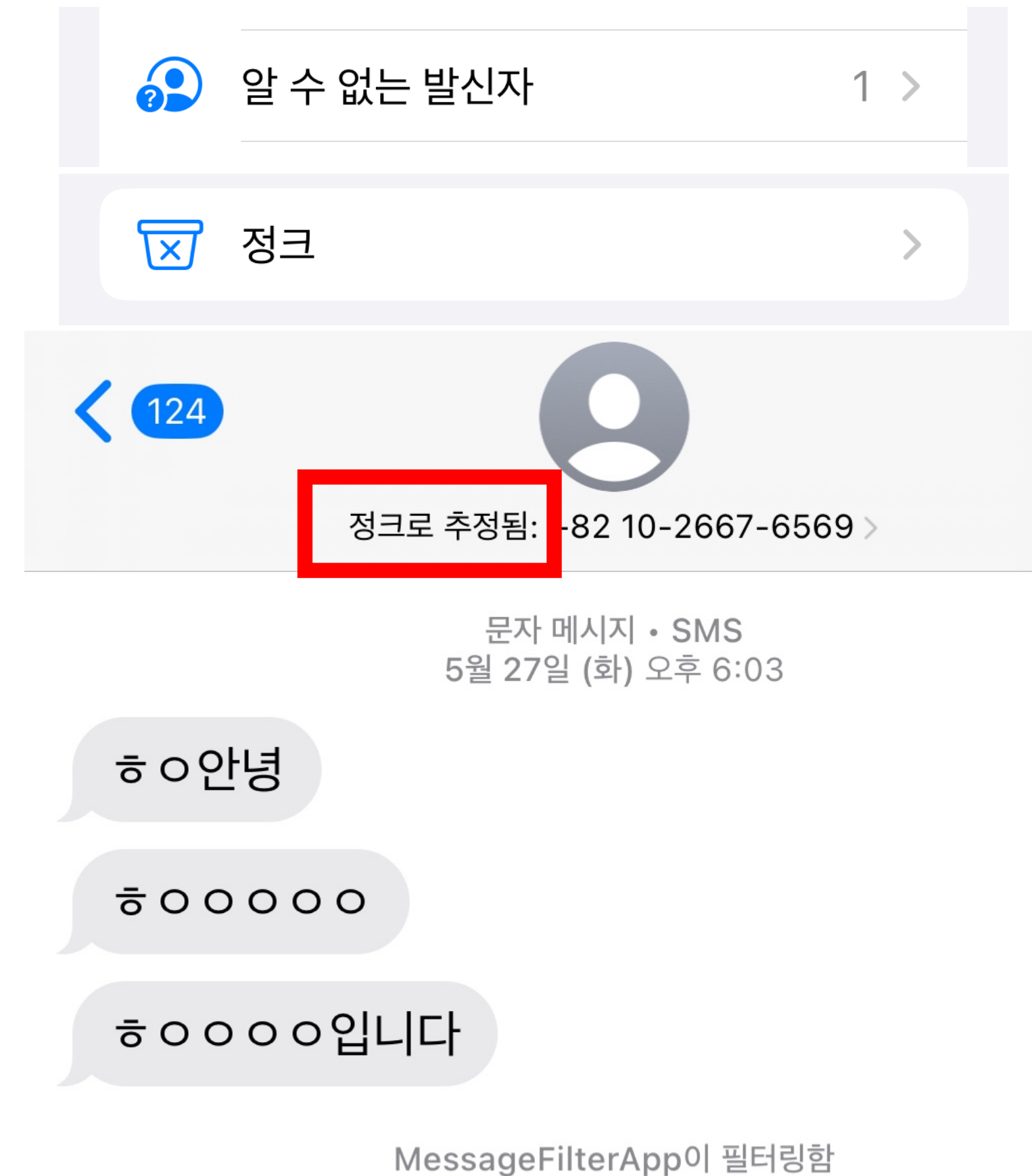
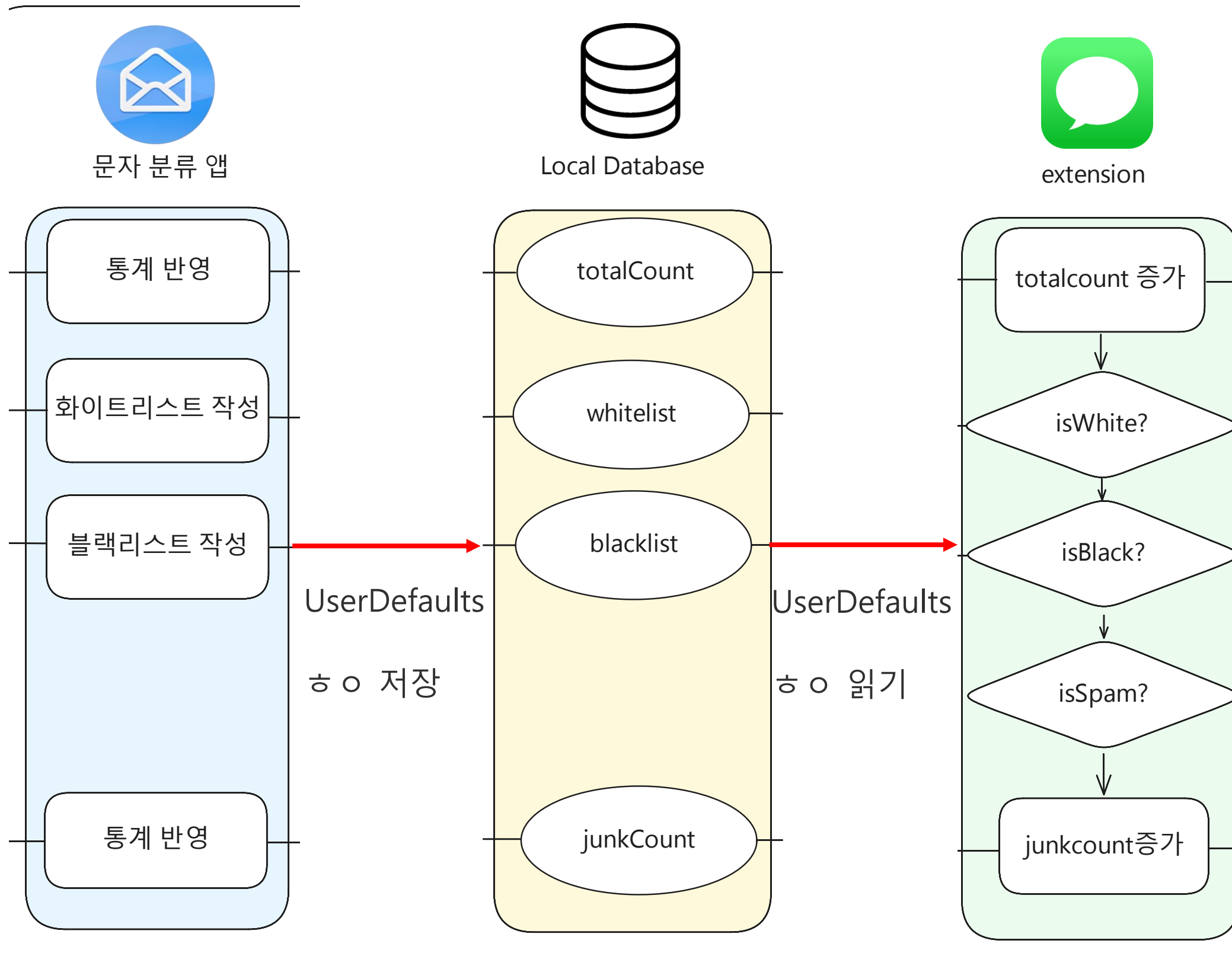
```
private func offlineAction(  
    for queryRequest: IMessageFilterQueryRequest  
    ) -> (IMessageFilterAction, IMessageFilterSubAction) {  
        guard let messageBody = queryRequest.messageBody else {  
            return (.none, .none)  
        }  
  
        let sender = queryRequest.sender ?? ""  
  
        if isWhitelisted(messageBody: messageBody, sender: sender) {  
            return (.allow, .none)  
        }  
  
        if isBlacklisted(messageBody: messageBody, sender: sender) {  
            return (.junk, .none)  
        }  
  
        if isSpamMessage(messageBody: messageBody, sender: sender) {  
            return (.junk, .none)  
        }  
  
        return (.allow, .none)  
    }  
}
```



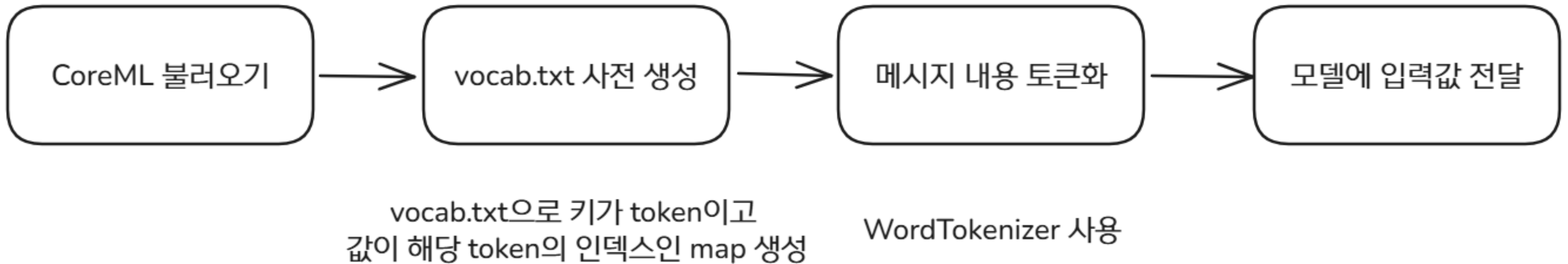
Local Database



iOS 주요 구현 내용 - 기술요소



iOS 주요 구현 내용 - 기술요소



iOS 통계 업데이트 문제

목표 :

문자(queryRequest)를 받으면 활성화되는 handle 함수에 totalCount 공유변수를 쓰고
스팸으로 판단되면 junkCount 공유변수를 쓰게하자

문제 :

totalCount와 junkCount가 ui에 반영되지 않음

구성 :

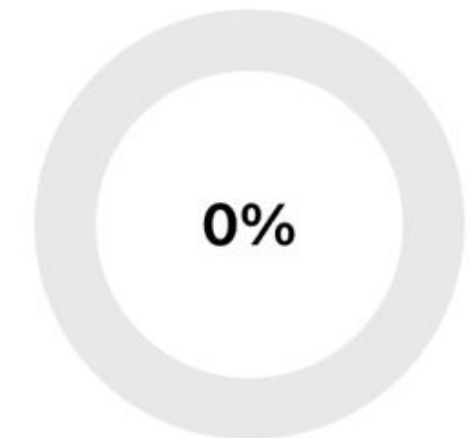
App Group을 활용한 UserDefaults(suiteName: "group.com.xxx.xxx") 에 저장

App 측) UserDefaults 변수를 읽어 UI에 통계값 표시

Extension 측) 특정 상황에 UserDefaults의 변수를 업데이트

전체 문자 0건 중

스팸 문자 0건을 차단했어요



iOS 통계 업데이트 문제

=> UI 업데이트 잘못된가?

문제의 변수들(totalCount, junkCount) 제외 다른 변수들은 ui refresh 성공

=> UserDefaults 쓰기 잘못된가?

Extension 밖 App 쪽에서 같은 코드를 실행했더니 UserDefaults 업데이트 성공

=> Extension 문제인가?

같은 함수 안에 있는 blacklist UserDefaults 변수, whitelist UserDefaults 변수 읽기 성공

iOS 통계 업데이트 문제

[현 상황]

App에서 UserDefaults 쓰고 Extension에서 UserDefaults읽기 가능

Extension에서 UserDefaults쓰고 App에서 UserDefaults 읽기 불가

결론 => 메시지 보안 때문에 MessageFilterExtension에서 쓰기 작업이 불가능한게 아닐까?

iOS AI 성능 문제

목표 :

float32->float16 변환으로 인한 정확도 감소를 반영하여 정확도 70% 이상의 AI 성능을 목표로 함

문제 :

CoreML 의 정확도가 매우 낮음

구성 :

WordPieceToknizer) vocab 사전을 생성하고 메시지를 받아서 토큰화

extension) WordPieceTokenizer에 접근해서 vocab 사전을 로드하고 메시지 내용을 토큰화

iOS AI 성능 문제

=> AI 연동 문제인가?

성능 개선 시도 이전에 AI 연동이 성공적이었던 것을 확인함

=> WordPieceTokenizer 코드 문제인가?

동일한 코드를 Extension이 아닌 APP에서 시도했을 때 높은 정확도를 보였음

=> WordPieceTokenizer 연동 문제인가?

WordPieceTokenizer 안의 함수에 대한 접근이 성공적임을 확인함

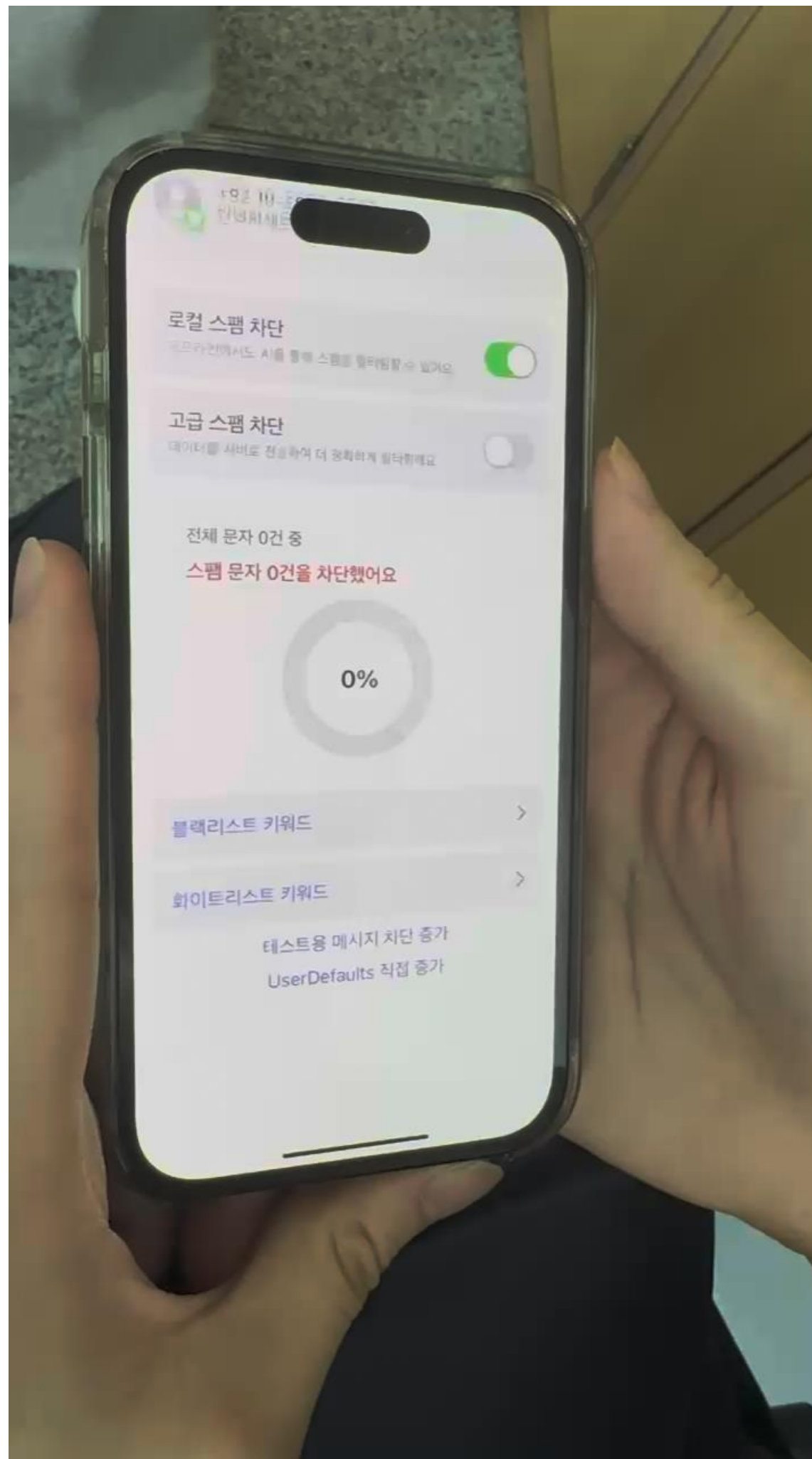
iOS AI 성능 문제

[현 상황]

동일한 코드임에도 불구하고 extension에서만 정확도가 낮음

결론 => 더 알아보기

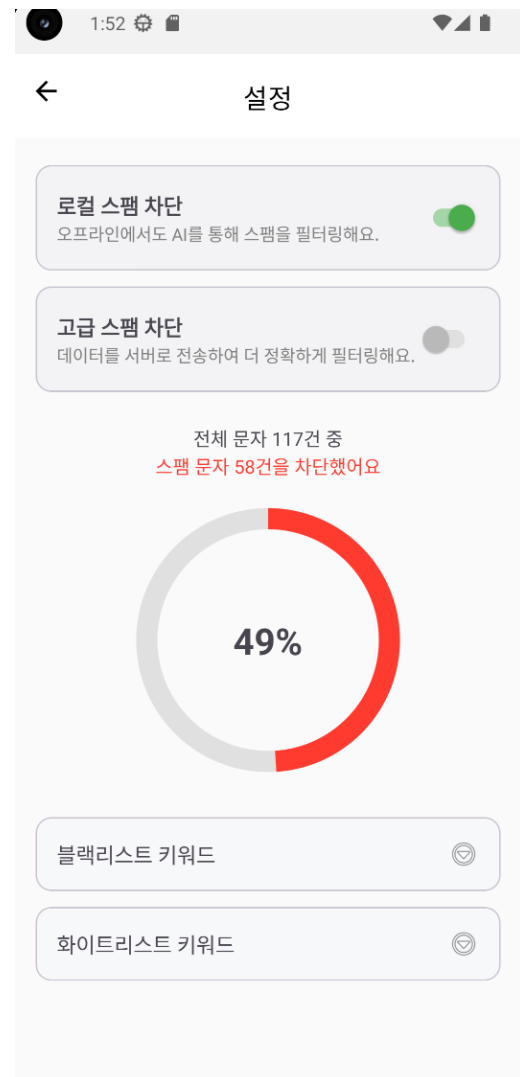
iOS 시연 영상



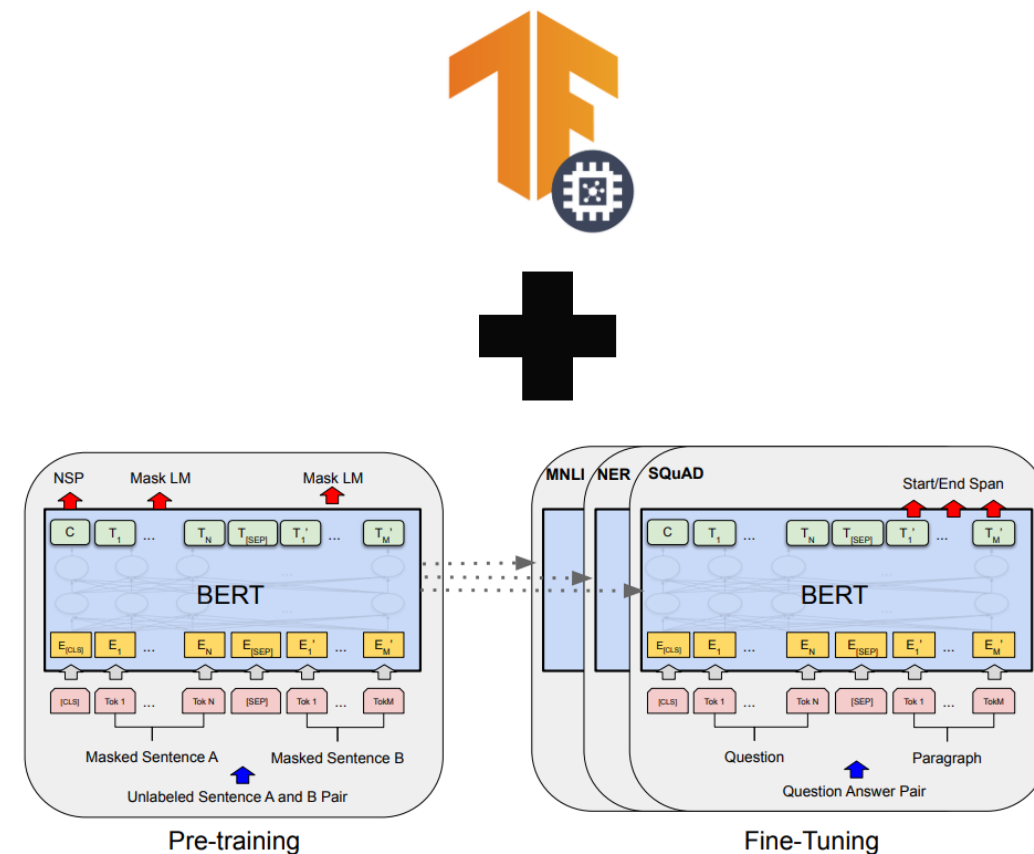
안드로이드

1. 시스템 하이라이트
2. 개발 환경 및 사용 기술
3. 아키텍처 & 흐름도
4. 세부 구현 내용

안드로이드 - 시스템 하이라이트



실시간 차단
통계 + 원형
그래프



TFLite 와
KoBERT의 결합

블랙리스트 키워드

화이트리스트 키워드

블랙리스트 키워드

키워드 입력 +

명칭이 ✗

바보 ✗

죽어버려 ✗

화이트리스트 키워드

키워드 입력 +

뚝뚝이 ✗

사랑해 ✗

실시간 키워드
화이트 / 블랙 리스트

안드로이드 - 개발 환경

호환성

IDE



Android Studio Giraffe

언어 & 데이터 쿼리



Kotlin 1.8.21 (JVM 11)
SQL

```
android {  
    namespace = "com.example.smsapp"  
    compileSdk = 35  
  
    defaultConfig {  
        applicationId = "com.example.smsapp"  
        minSdk = 26  
        targetSdk = 35  
        versionCode = 1  
        versionName = "1.0"  
    }  
  
    buildFeatures { viewBinding = true }  
  
    compileOptions {  
        sourceCompatibility = JavaVersion.VERSION_11  
        targetCompatibility = JavaVersion.VERSION_11  
    }  
    kotlinOptions { jvmTarget = "11" }  
}
```

빌드 SDK

안드로이드 - 사용 라이브러리

화면 전환
프레임 워크



Jetpack Navigation

로컬 DB

Room



Room (kapt)

디자인



Material 3

AI 엔진



TFLite

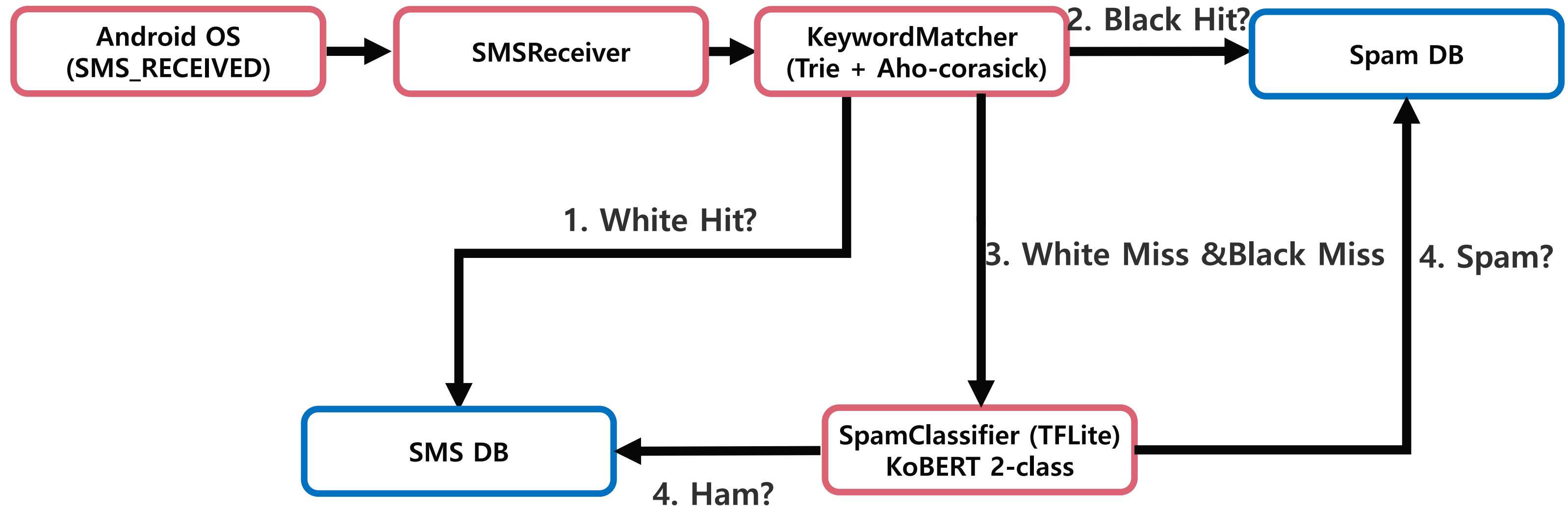
비동기 처리



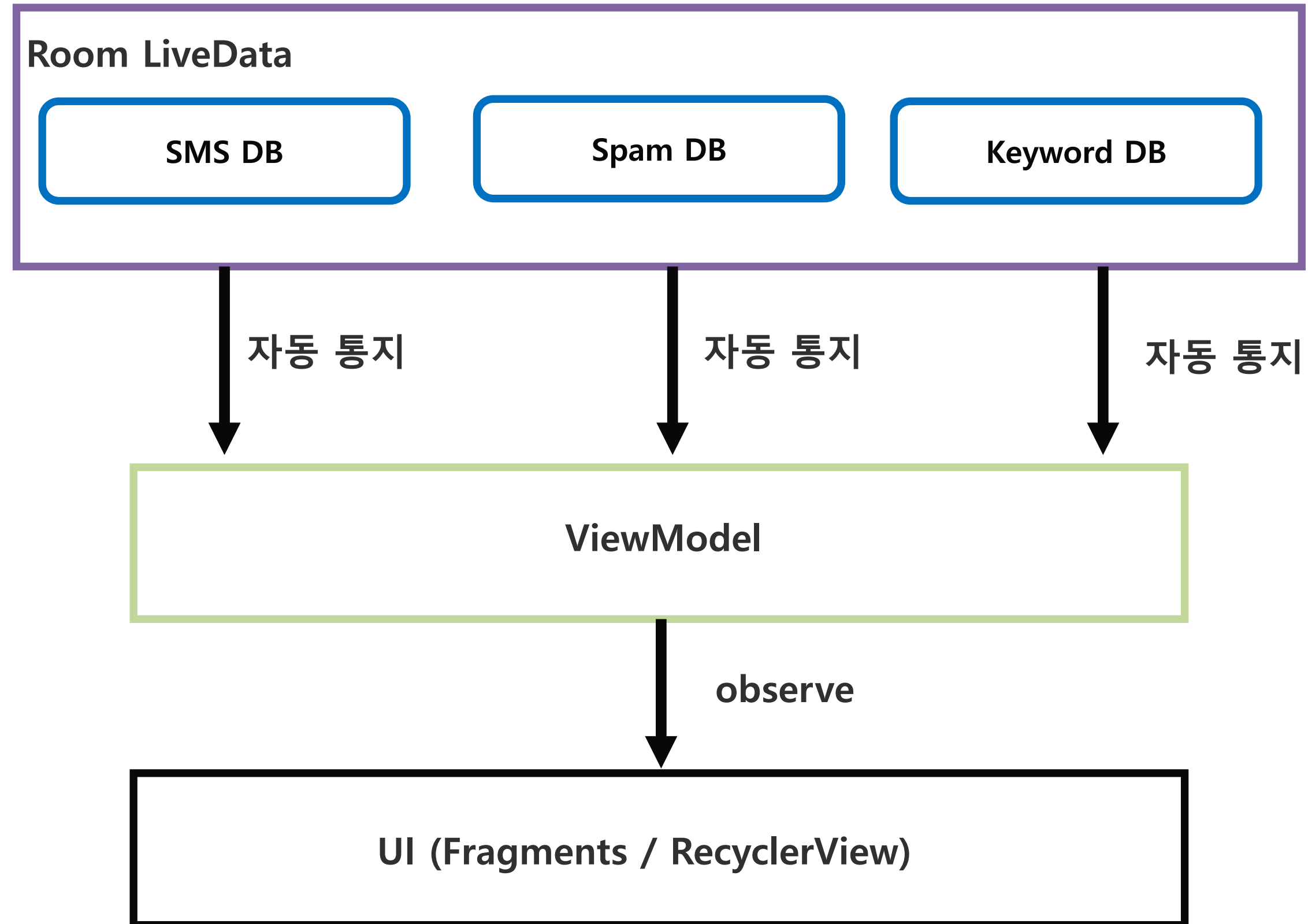
Kotlin
Coroutines

Kotlin Coroutines

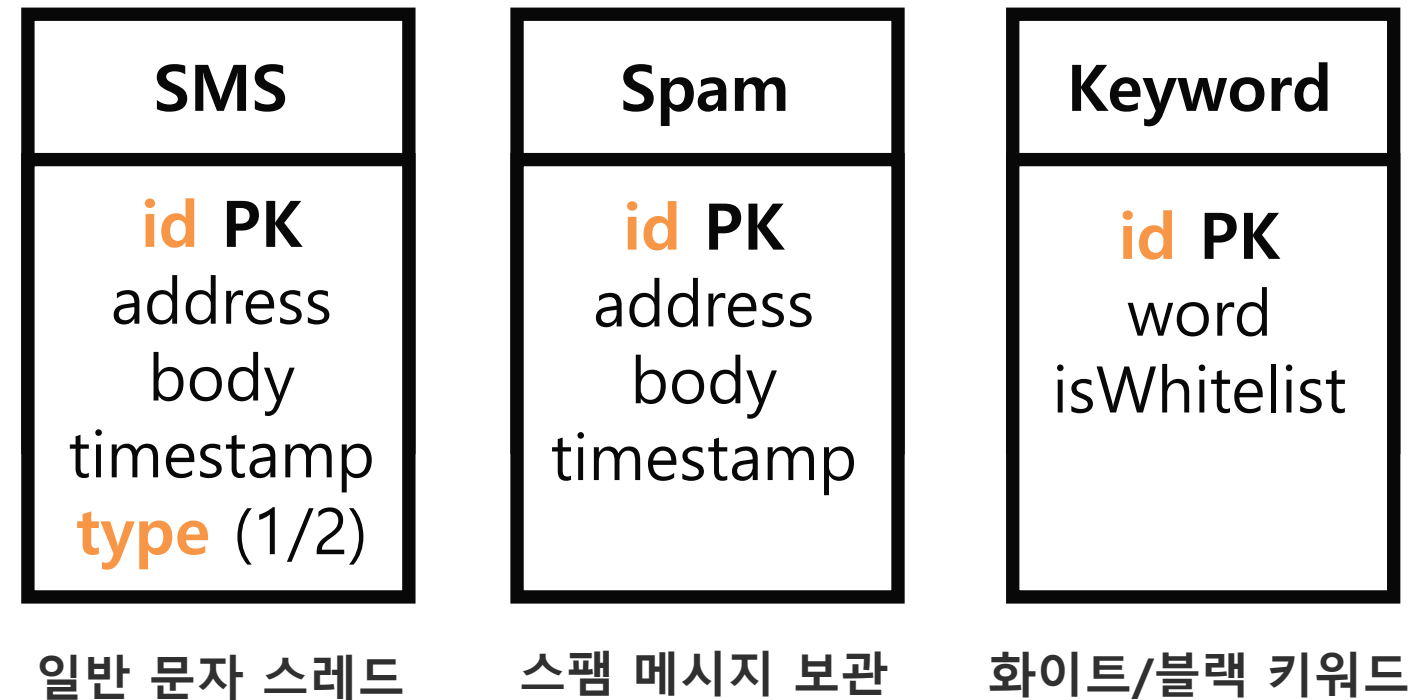
안드로이드 - 흐름도



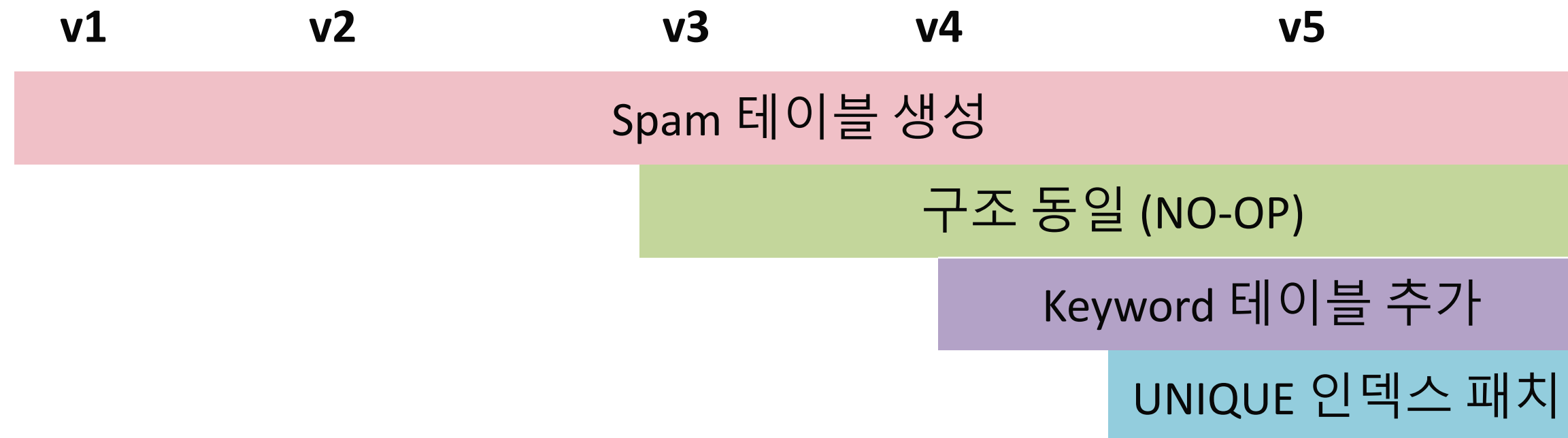
안드로이드 - 흐름도



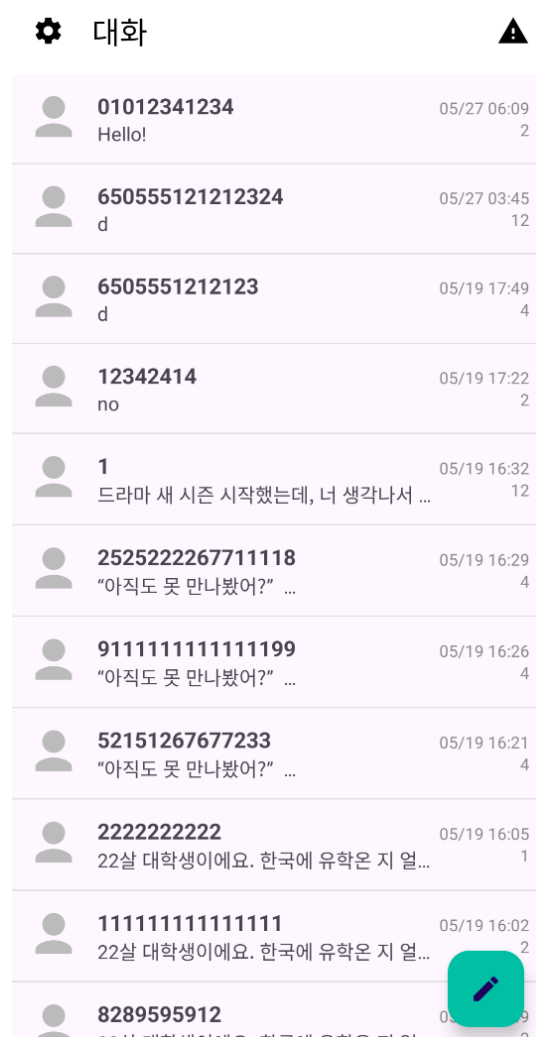
안드로이드 - 데이터 모델과 Room DB 설계



Migration Timeline

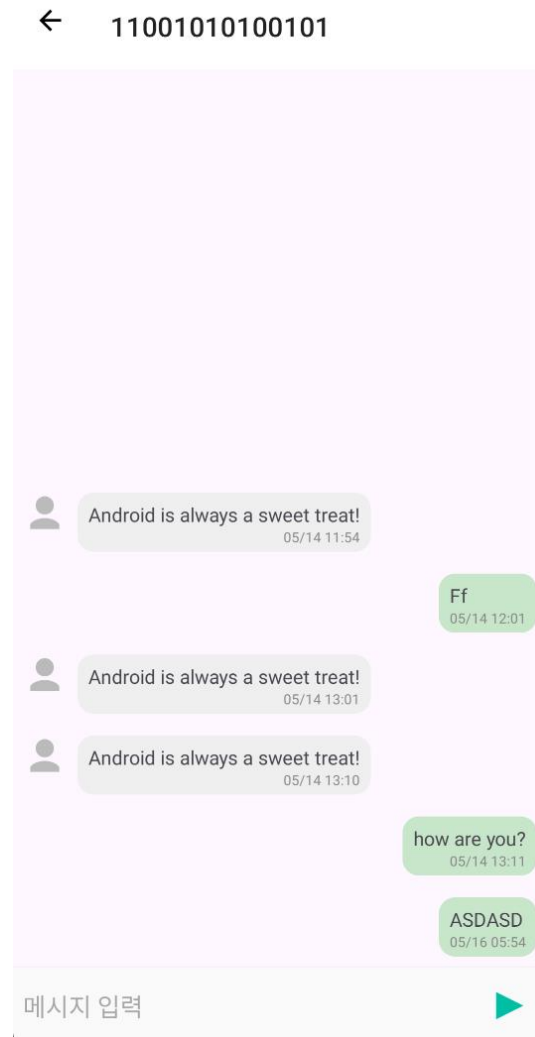


안드로이드 - UI / UX



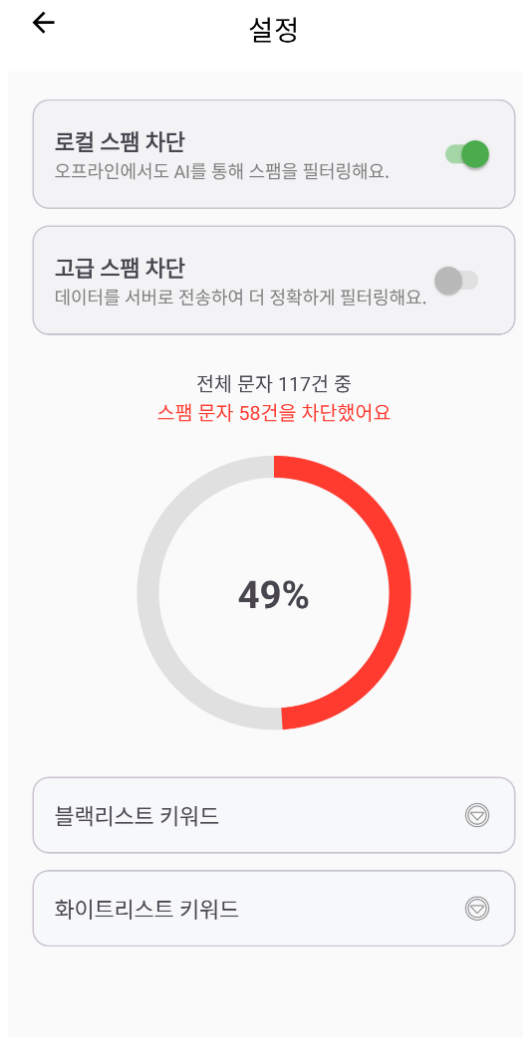
대화 목록

- Material3 Toolbar
- RecyclerView
Swipe-To-Delete



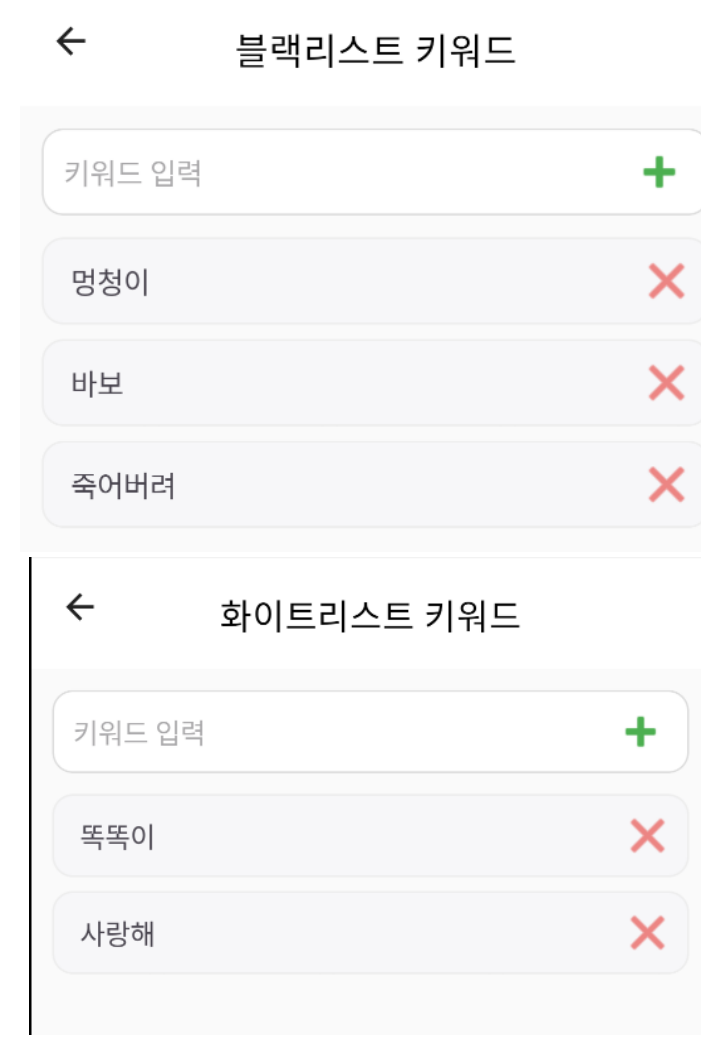
스레드

- 버블 컬러(M3 Surface)
- ReverseLayout



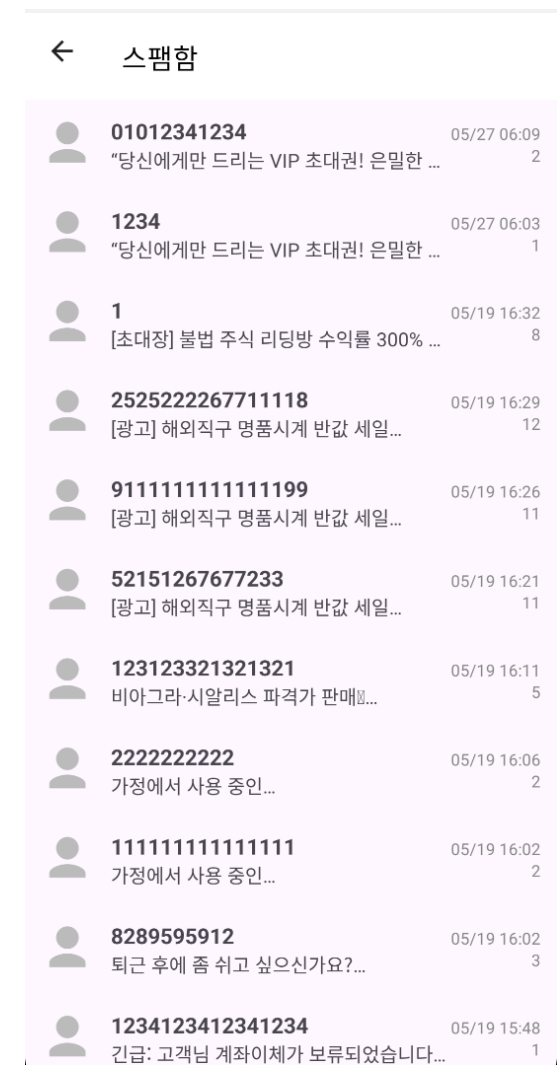
설정

- 통계 Ring
- Local-AI Switch



키워드

- Real-time LiveData
- KeywordMatcher.invalidate()



스팸함

- 스팸만 분리 보관
- 동일 UI 재사용

안드로이드 - 백엔드 (수신 파이프라인)

```
class SMSReceiver : BroadcastReceiver() {

    override fun onReceive(ctx: Context, intent: Intent) {
        if (intent.action != Telephony.Sms.Intents.SMS_RECEIVED_ACTION) return

        //여러 segment 합치기
        val parts = Telephony.Sms.Intents.getMessagesFromIntent(intent)
        if (parts.isEmpty()) return
        val address = parts[0].originatingAddress ?: return
        val body = buildString { parts.forEach { append(it.messageBody) } }
        val timestamp = System.currentTimeMillis()

        val db = SmsDatabase.get(ctx)
        val smsDao = db.smsDao()
        val spamDao = db.spamDao()

        // IO 코루틴
        CoroutineScope(Dispatchers.IO).launch {
            //워드 매처 초기화
            KeywordMatcher.init(ctx)

            // 화이트/블랙 키워드 우선 검사
            val res = KeywordMatcher.match(body)
            val localOn = AppPrefs.isLocalFilterEnabled(ctx)
            val isSpam = when {
                res.hasWhite <--> false
                res.hasBlack <--> true
                localOn && SpamClassifier.isSpam(body) <--> true
                else <--> false
            }

            // 결과에 따라 DB 저장
            if (isSpam) {
                spamDao.insert(
                    SpamEntity(address = address, body = body, timestamp = timestamp)
                )
            } else {
                smsDao.insert(
                    SmsEntity(address = address, body = body,
                        timestamp = timestamp, type = 1) // 1 = 수신
                )
            }
        }
    }
}
```

- BroadCast 필터 -> SMS_RECEIVED
- Multipart 조각 -> 본문 병합
- Coroutine(IO) -> 메인 차단 x
- KeywordMatcher.init() -> 중복이라면 재활용
- White Hit -> 통과
- White Miss / Black Hit -> 즉시 차단
- Both Miss? -> TFLite KoBERT 추론 호출

SMS 권한은 Google Play 정책에 따라 READ/RECEIVE/SEND 필수

안드로이드 - 백엔드 (키워드 매칭)

```
// Aho-Corasick Fail link
private fun buildAC(root: Node) {
    val q: ArrayDeque<Node> = ArrayDeque()
    root.fail = root
    root.next.values.forEach { child ->
        child.fail = root
        q.add(child)
    }
    while (q.isNotEmpty()) {
        val cur = q.removeFirst()
        for ((c, nxt) in cur.next) {
            var f = cur.fail
            while (f != root && c !in f!!.next) f = f.fail
            nxt.fail = if (c in f!!.next) f.next[c]!! else root
            nxt.isEnd = nxt.isEnd || nxt.fail!!.isEnd // 출력 병합
            q.add(nxt)
        }
    }
}
```

- 두 Trie(White / Black) 분리
- Fail link의 구성의 시간 복잡도 -> $O(\text{nodes})$
- 검색의 시간 복잡도 -> $O(N)$
- White Hit 시 -> Black Check · AI Check skip

안드로이드 - 백엔드 (AI 분류 호출)

```
private fun runModel(text: String): Pair<Float, Float> {
    val norm = Normalizer.normalize(text, Normalizer.Form.NFKC)
        .toLowerCase(Locale.getDefault())
    val tokens = tokenize(norm)

    val cls = vocab["[CLS]"]!!.toLong()
    val sep = vocab["[SEP]"]!!.toLong()
    val unk = vocab["[UNK]"]!!.toLong()

    val vocabSize = vocab.size.toLong()
    val ids = LongArray(seqLen) { 0L }
    val mask = LongArray(seqLen) { 0L }

    var i = 0
    ids[i] = cls.coerceIn(0L, vocabSize - 1); mask[i] = 1L; i++
    for (tok in tokens) {
        if (i >= seqLen - 1) break
        val rawId = vocab[tok]?.toLong() ?: unk
        val safeId = rawId.coerceIn(0L, vocabSize - 1)
        ids[i] = safeId
        mask[i] = 1L
        i++
    }
    ids[i] = sep.coerceIn(0L, vocabSize - 1)
    mask[i] = 1L

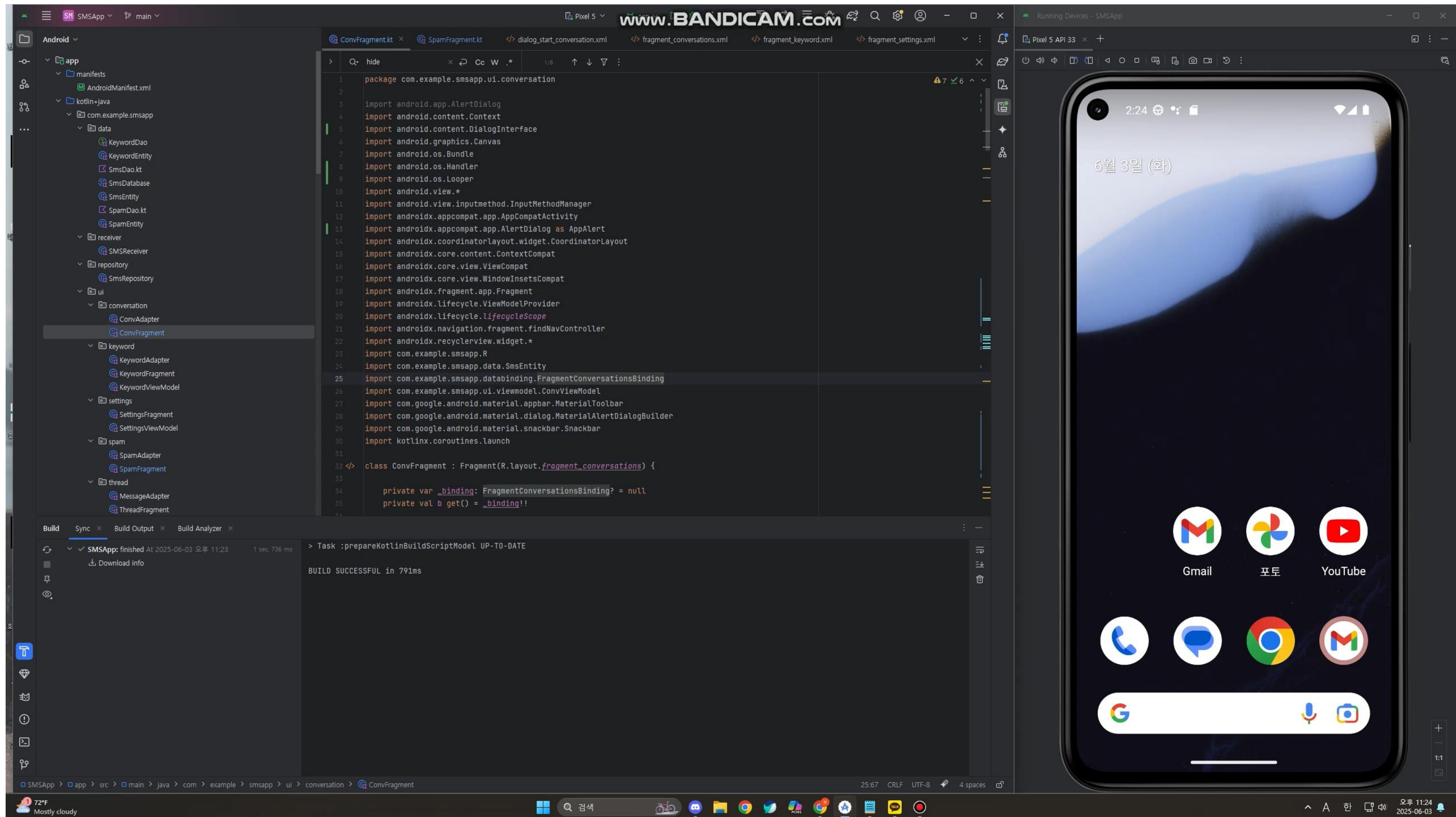
    val inputs = arrayOf(arrayOf(ids), arrayOf(mask))
    val out = Array(1) { FloatArray(2) }

    interpreter.runForMultipleInputsOutputs(inputs, mapOf(0 to out))

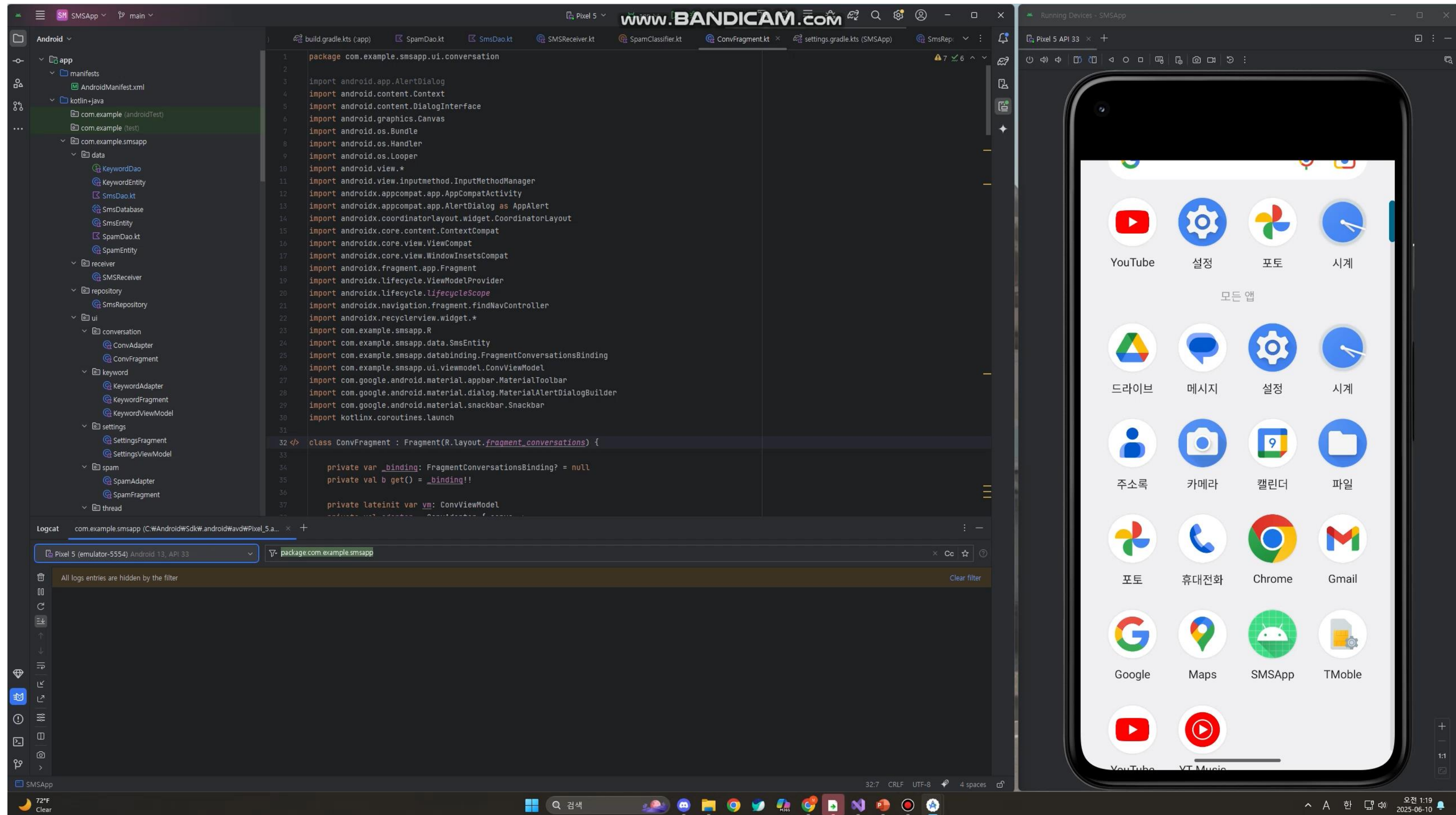
    // logit_ham 에서 logit_spam으로 리턴
    return out[0][0] to out[0][1]
}
```

- FP16모델 -> 6.4MB (모바일 최적)
- 시퀀스 길이 128 -> 일반 SMS 100% 수용
- 토큰화(id · mask) -> 두 개 Tensor 입력
- Softmax 함수에서 Spam 확률 ≥ 0.60 -> 차단
- 100% 오프라인 동작

시연 영상

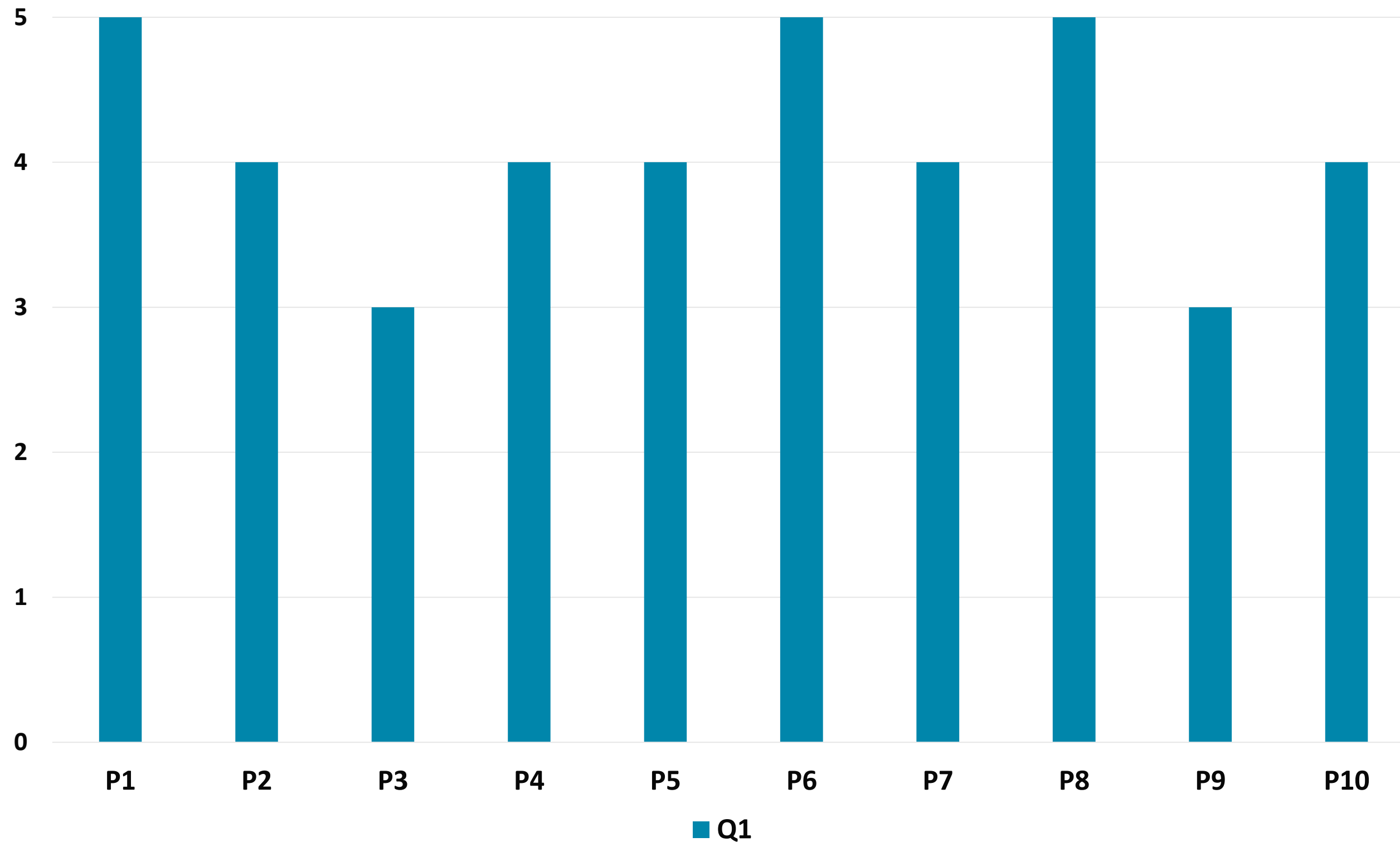


시연 영상



베타테스팅 결과

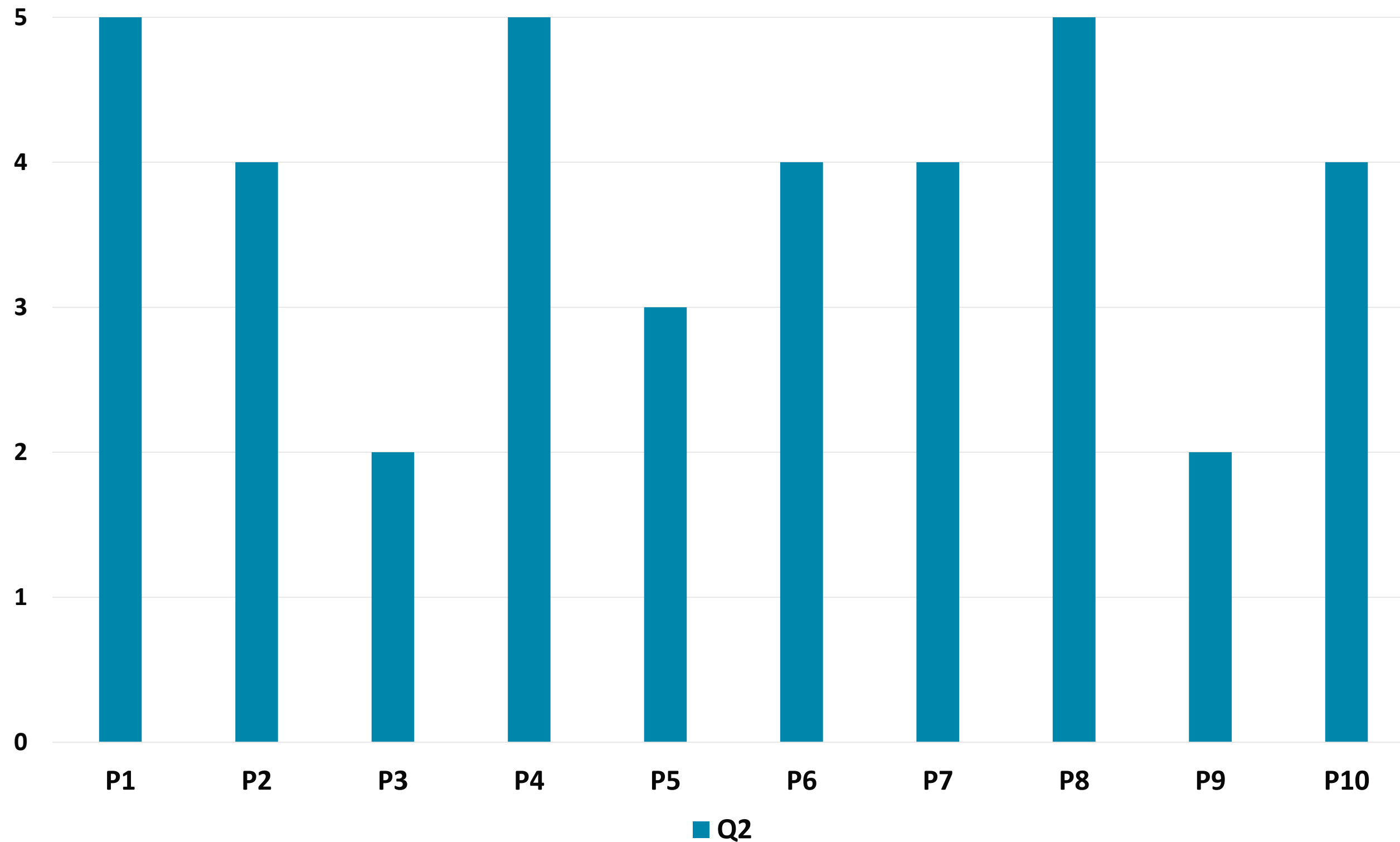
Q1 - 앱이 스팸 메시지를 정확하게 걸러냈다고 생각하시나요?



AVG : 4.1

베타테스팅 결과

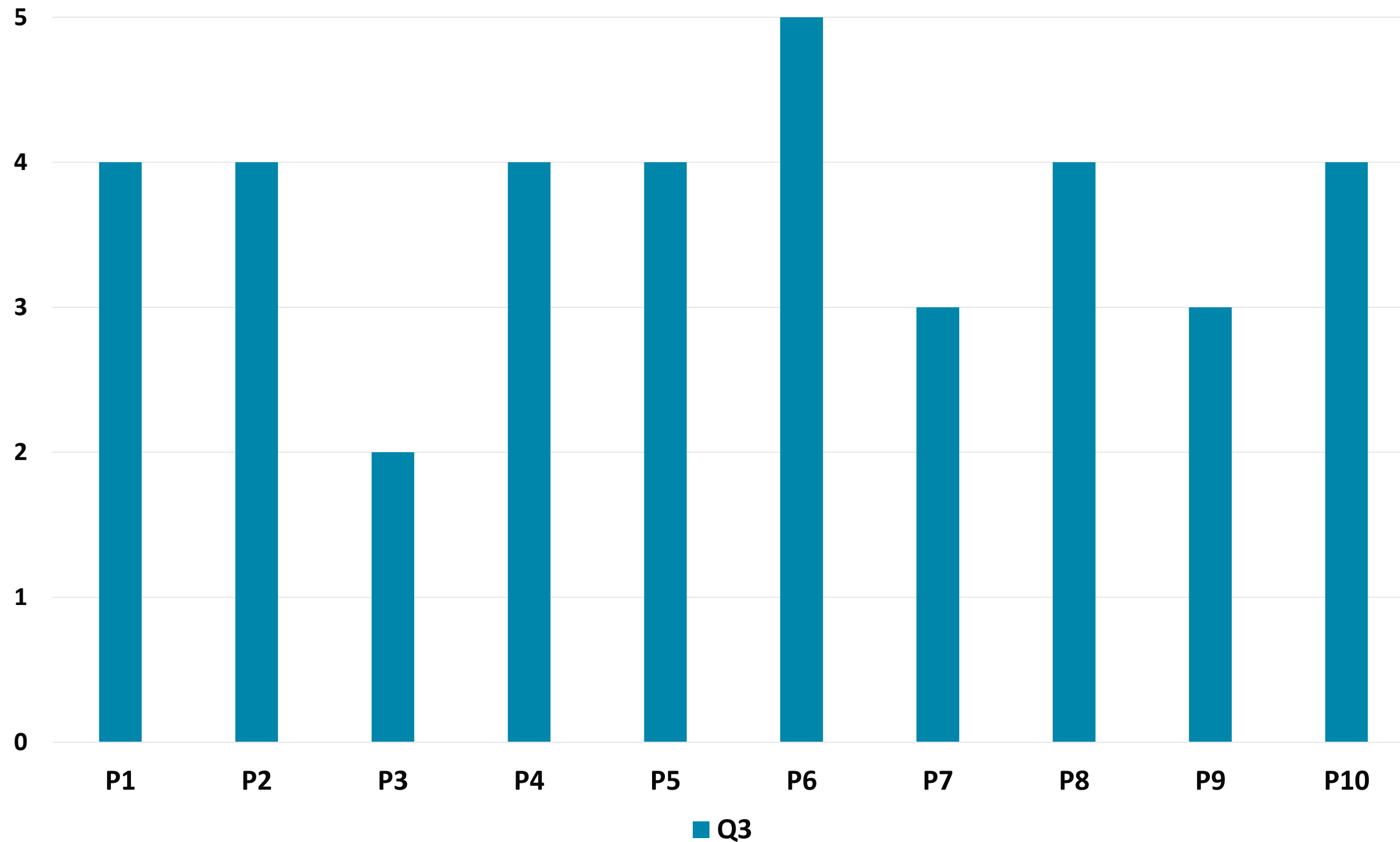
Q2 - 새 메시지가 도착했을 때 목록이 즉시 갱신되었나요?



AVG : 3.7

베타테스팅 결과

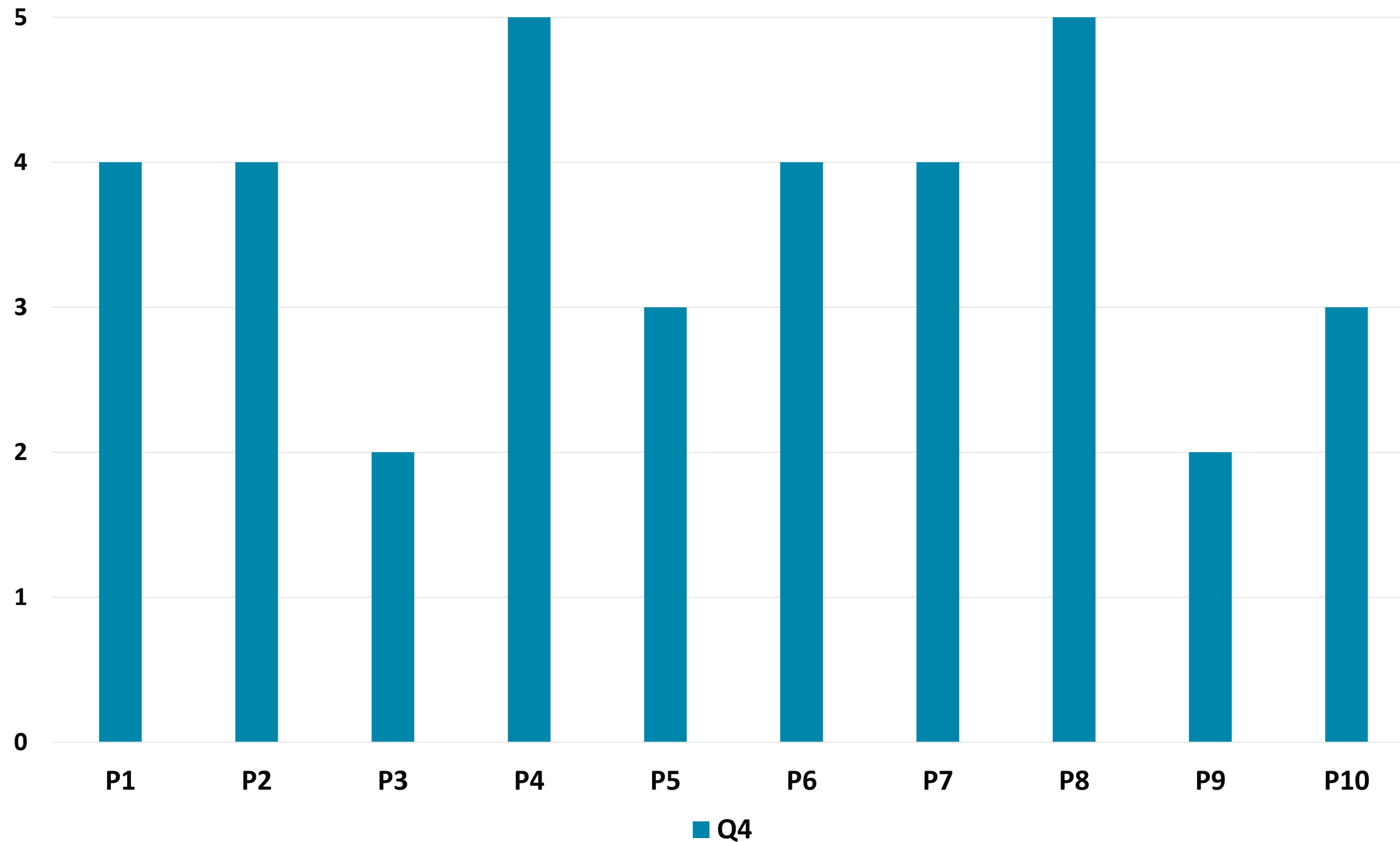
Q3 - 화면 배치·색상 등 UI가 직관적이었나요?



AVG : 3.7

베타테스팅 결과

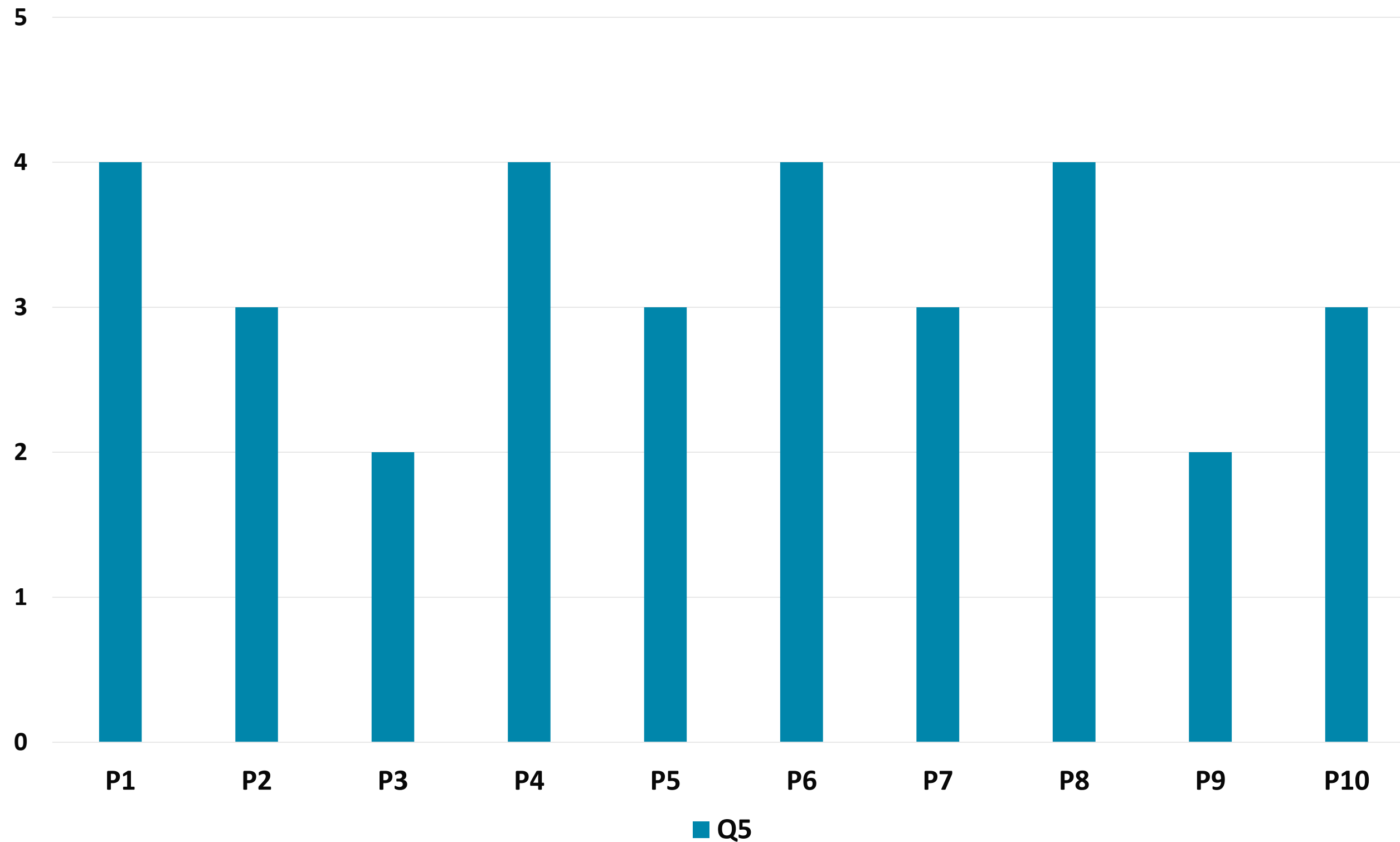
Q4 - 화이트·블랙 키워드를 추가/삭제하는 과정이 간단했나요?



AVG : 3.6

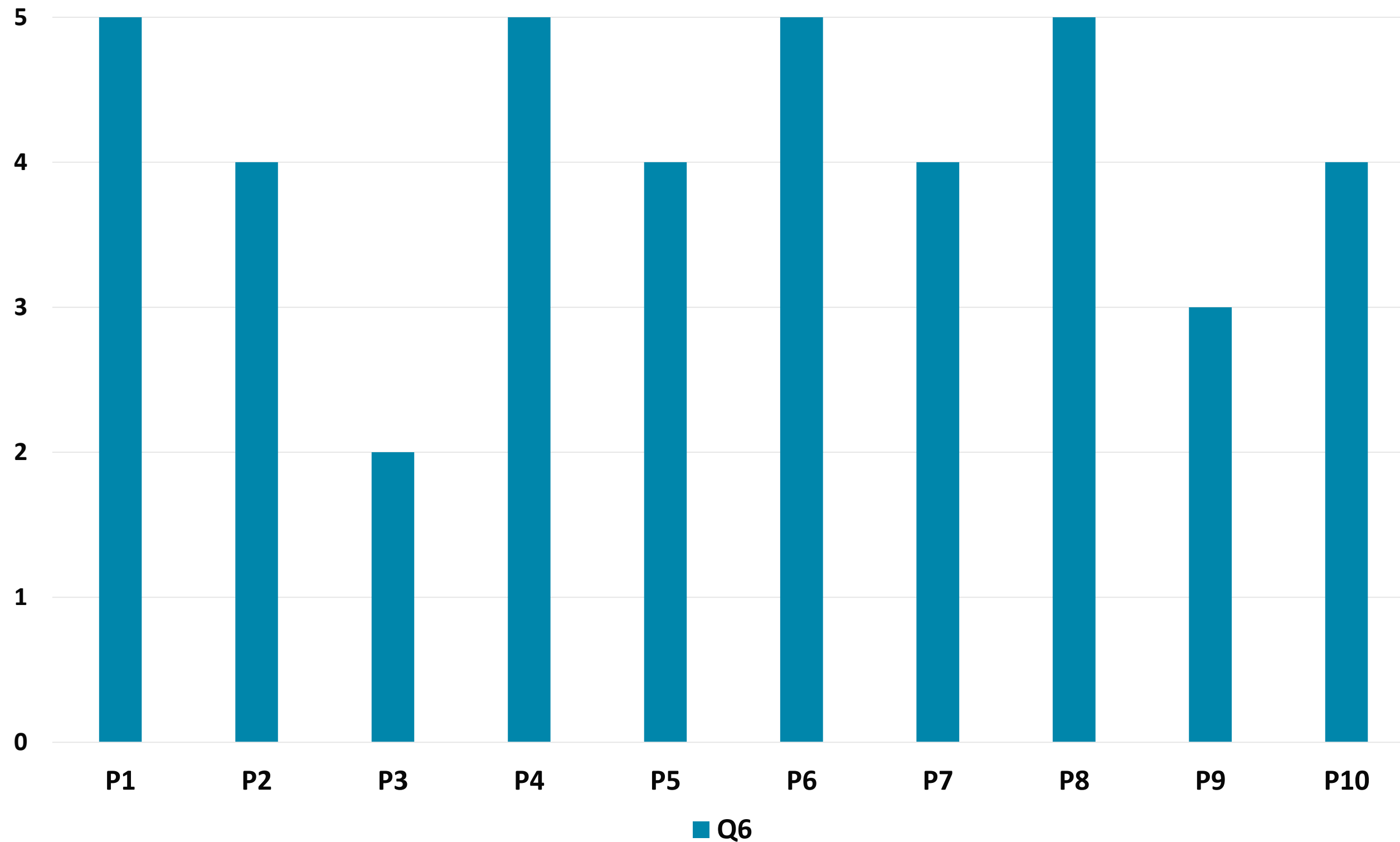
베타테스팅 결과

Q5 - 설정 화면(통계·토글)을 이해하기 쉬웠나요?



베타테스팅 결과

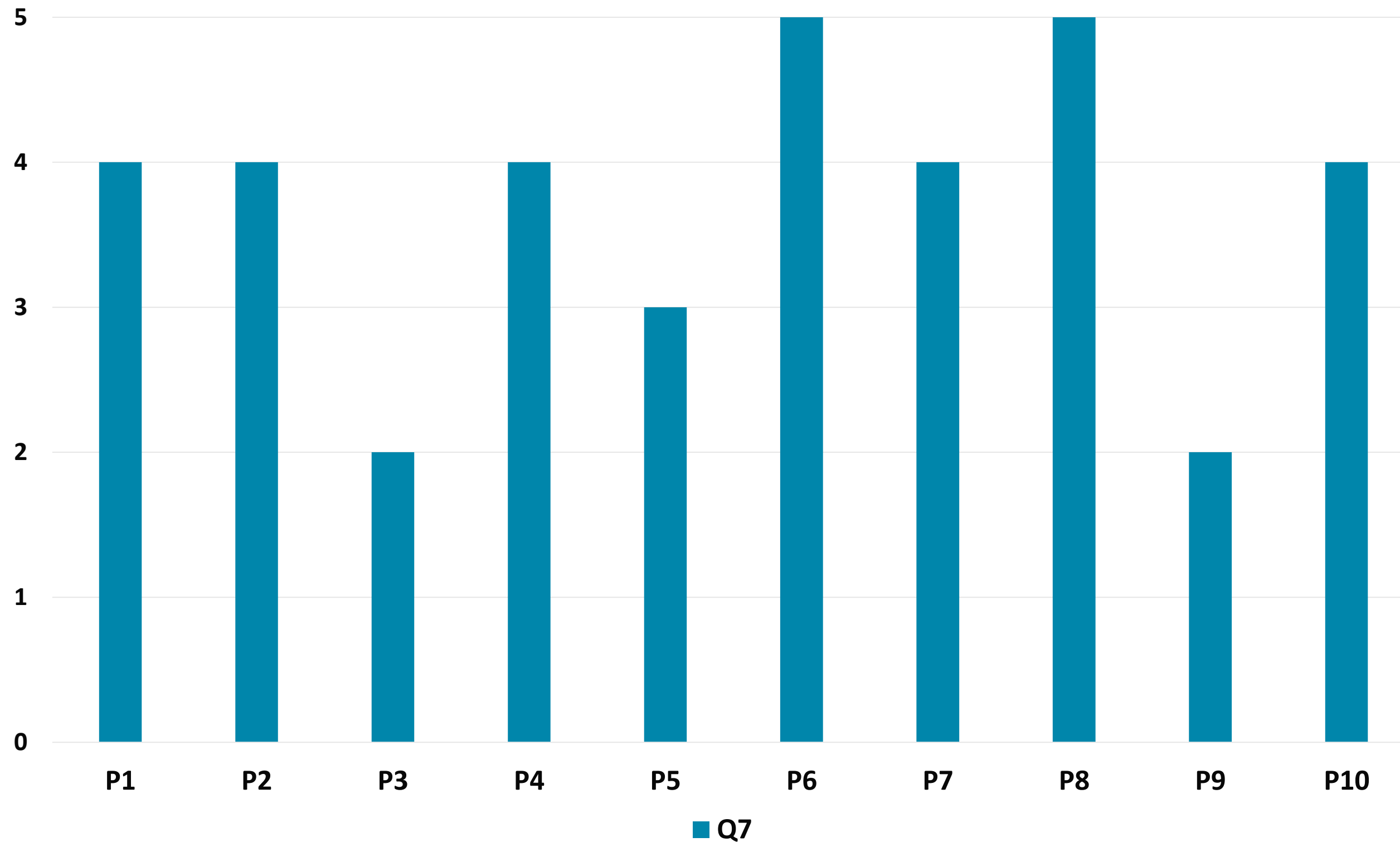
Q6 - 전반적으로 이 앱 사용 경험에 만족하시나요?



AVG : 4.1

베타테스팅 결과

Q7 - 이 앱을 기본 문자 앱으로 계속 사용할 의향이 있나요?



AVG : 3.7

테스팅 결과

정확도

대부분의 참가자가 필터 정확도는 「기대 이상」이라고 평가 하였음
정상 문자 까지 스팸함으로 가는 경우는 드물게 1~2건에 그쳤음

반응 속도

「문자가 오자마자 목록이 새로 고침 된다」는 긍정 의견이 많았음.
다만, 앱 최초 실행시 모델을 불러오는 구간에서 「멈춘 것 같다」라는 불만이 일부 제기 되었음

UI / UX

화면 배치 / 색상은 「심플해서 금방 익숙해진다」가 다수였다.
하지만 설정 메뉴가 살짝 복잡하거나, 다크 모드가 없다는 지적이 반복되었음

키워드 관리

기본 기능은 잘 동작 했으나, 「여러 키워드를 한꺼번에 넣기엔 너무 번거롭다」
이라는 사용성 이슈가 확인 되었음

피드백

정확도

- OTP / 알림 패턴의 화이트 리스트를 추가

- Splash 화면에 로딩 프로그레스 삽입

반응 속도

- 모델 lazy-load : 첫 SMS 수신 시만 초기화
- FP16 -> INT8 양자화 (RAM -65 %, 로딩 속도 +70 %)

UI / UX

- Theme.Material3.DayNight 완전 지원 하도록 하기
- 고급 스팸 차단 섹션 접기 / 도움말 툴팁 추가

키워드 관리

- 줄바꿈이나 심표를 구분하는 다중 입력 지원
- CSV 가져오기 & 내보내기 기능 추가

향후 개선 계획 및 기대 효과 (팀)

한계점

- ◆ 온디바이스 AI는 서버 기반 AI 대비 높은 필터링 성능을 기대하기 어려움

개선 계획

- ◆ 클라우드에 고성능 AI를 배치하여, 더 정확한 필터링을 제공함

기대 효과

- ◆ 사용자 데이터를 수집하여 피드백 형식으로 성능을 높일 수 있음
- ◆ 온디바이스 형식과 서버 형식을 모두 사용 하여 사용자에게 더 나은 서비스 경험을 제공할 수 있음